Syrian Private University
Faculty of Artificial Intelligence Engineering
Intelligent Information Security Systems Engineering

Design and Implementation of a Security Filter for Detecting Malicious File Uploads in Web Applications

A project prepared to obtain a Bachelor's degree in Artificial Intelligence Engineering, specializing in Intelligent Information Security Systems Engineering — Senior Project II

Prepared by:
Ahmad Khalluof
Talal Alhelou

Supervised by:
Dr. Wasim Juneidi
Eng. Mohammed Yamen Hallak

August 2026

Abstract

File upload is one of the most commonly exposed and most frequently abused features of modern web applications. A single endpoint that accepts user-supplied files must simultaneously defend against a wide spectrum of attacks — from trivial extension spoofing, through polyglot files that are at once a valid image and a valid script, to malicious documents whose payloads are concealed behind layered stream compression. Conventional upload validators apply a single check — an extension whitelist, a MIME sniff, or one signature scan — and are therefore defeated by any input crafted to satisfy that one check while smuggling an executable payload past it.

This project presents SecureUploader, a defense-in-depth pipeline of nine ordered inspection layers that transforms an untrusted uploaded file into either a rejected verdict accompanied by a precise per-layer trace, or an accepted-but-neutralized artifact. Beyond detection, SecureUploader treats neutralization — the deterministic rewriting of a file to strip an embedded payload while preserving its legitimate content — as a first-class outcome and as a second evaluation metric alongside detection. The pipeline hardens several evasion classes that defeat single-check validators: nested-filter PDF compression, image-carried scripts, and active SVG content.

SecureUploader was evaluated on a 10,012-file corpus drawn from CIC-Evasive-PDFMal2022, achieving 95.48% recall, 93.05% precision, 93.54% accuracy, and a 94.25% F1 score, and was further stress-tested against a hand-built adversarial corpus of real polyglots and evasive fixtures, compared side-by-side against five reference validators. A dedicated study on a pixel-domain steganography dataset showed that a deterministic least-significant-bit clearing step neutralizes 100% of LSB-carried payloads while remaining visually imperceptible (PSNR above 49 dB). Three self-bypass vulnerabilities discovered during development were diagnosed and fixed, and every verdict is accompanied by a reproducible, per-layer evidence trail — providing the transparency that opaque machine-learning detectors lack.

Keywords: file-upload security, defense in depth, polyglot files, evasion resistance, content neutralization, magic-byte validation, YARA signature scanning, PDF FlateDecode inflation, SVG sanitization, empirical evaluation.

إهداء

إلى والديّ الكريمين، اللذين كانا سندي ودعمي في كل خطوة...

إلى أساتذتي الكرام، على جهدهم وتوجيههم طوال مسيرة هذا البحث...

إلى كل من أسهم في إثراء مسيرتنا العلمية بعلمه أو تشجيعه...

طلال الحلو ـــ أحمد خلوف

ملخص البحث

يُعدّ رفع الملفات (File Upload) من أكثر الميزات انتشاراً في تطبيقات الويب الحديثة، وفي الوقت نفسه من أكثرها استغلالاً من قبل المهاجمين. فنقطة النهاية (Endpoint) التي تقبل ملفات المستخدم يجب أن تتصدّى في آنٍ واحد لطيفٍ واسع من الهجمات؛ بدءاً من انتحال الامتداد البسيط، مروراً بملفات الـ Polyglot، وصولاً إلى المستندات الخبيثة التي تُخفي حمولتها خلف طبقات متتالية من الضغط.

يقدّم هذا المشروع أداة SecureUploader، وهي خط دفاعٍ متعدد الطبقات مؤلّف من تسع طبقات فحص مرتّبة، يحوّل الملف غير الموثوق إلى إمّا رفضٍ مصحوبٍ بأثرٍ دقيق لكل طبقة، أو قبولٍ بعد تحييد الحمولة (Neutralization). جرى تقييم الأداة على 10012 ملفاً مأخوذة من CIC-Evasive-PDFMal2022، محقّقةً Recall 95.48% و Precision 93.05% و Accuracy 93.54% و F1 94.25%، ثم اختُبِرت على مجموعةٍ خصامية مبنيّة يدوياً. كما أظهرت دراسةٌ مخصّصة على مجموعة صور الإخفاء بمجال البكسل (LSB) أن خطوة تصفير البت الأقل أهمية تُبطِل 100% من الحمولات المخبّأة مع بقاء التشويه غير محسوس بصرياً (PSNR أعلى من 49 dB). وقد عولجت ثلاث ثغرات للتجاوز الذاتي اكتُشفت أثناء التطوير.

الكلمات المفتاحية: أمن رفع الملفات، الدفاع متعدد الطبقات، ملفات Polyglot، مقاومة التحايل، تحييد المحتوى، YARA، التقييم التجريبي.

Table of Contents

Abstract 2
إهداء 3
ملخص البحث 4
Table of Contents 5
List of Figures 10
List of Tables 11
List of Abbreviations and Technical Terms 12
Chapter One — Introduction 13
1.1 Introduction 14
1.2 Problem Statement 15
1.3 Research Objective 15
1.4 Research Questions 16
1.5 Scope and Assumptions 16
1.6 Practical Applications 17
1.6.1 Hardening Web-Application Upload Endpoints 17
1.6.2 Supporting Security Operations and Triage 17
1.6.3 Integration into Continuous-Delivery Pipelines 17
1.6.4 A Foundation for Research and Coursework 18
1.7 Challenges 18
1.8 Literature Review 19
1.8.1 Published Upload-Checking Systems (FileUploadChecker) 19
1.8.2 Evasion Taxonomies for File Upload (FUEL / Fuxploider-NG) 19
1.8.3 Empirical Study of Polyglot Files (Koch et al.) 20
1.8.4 Machine-Learning Detection of Malicious Documents 20
1.8.5 Re-encoding and Round-Trip Sanitization 20
1.8.6 The Research Gap 21
1.9 Structure of the Report 21
Chapter Two — Theoretical Background 22
2.1 Introduction 23
2.2 File Formats and How Parsers Read Them 23
2.2.1 JPEG Structure 24
2.2.2 PNG Structure 24
2.2.3 PDF Object Model 24

2.2.4 SVG and the XML Model 24
2.3 The File-Upload Attack Surface 25
2.3.1 HTTP Upload Mechanics 25
2.3.2 Storage, Processing, and Serving Paths 26
2.4 A Taxonomy of File-Upload Attacks 26
2.4.1 Extension Spoofing 27
2.4.2 MIME / Content-Type Spoofing 27
2.4.3 Web Shells 28
2.4.4 Polyglot Uploads 28
2.4.5 Stored Cross-Site Scripting via Uploads 28
2.4.6 XXE and SSRF via XML-Based Formats 28
2.4.7 Compression and Decompression Attacks 28
2.5 Polyglot Files in Depth 29
2.5.1 A Worked Polyglot Example 30
2.6 Nested-Filter and Compression Concealment 30
2.6.1 Decompression Bombs and Expansion Ratios 31
2.7 Active Content in Vector Formats 32
2.8 Inspection Building Blocks 32
2.8.1 Extension Whitelisting 32
2.8.2 Magic-Byte / Signature Validation 32
2.8.3 Header–Content Matching 32
2.8.4 YARA Signature Scanning 32
2.8.5 Content Re-encoding and Sanitization 33
2.9 Neutralization as a Distinct Outcome 33
2.10 Mapping Evasion Techniques to Defenses 33
2.11 Comparison of Detection Strategies 34
2.12 Chapter Summary 35
Chapter Three — Methodology and Environment 36
3.1 Introduction 37
3.2 Research Methodology 37
3.3 Development Platform and Language 37
3.4 Libraries, Tools, and Environment 38
3.5 Threat Model 38
3.6 Evaluation Datasets and Corpus Construction 39

3.6.1 The Public Corpus (CIC-Evasive-PDFMal2022) 40
3.6.2 Real Polyglots and the Download Route 40
3.6.3 Benign Corpus and Inert Fixtures 40
3.7 Containment and the Handling of Real Malware 41
3.8 Accepted File Types and Threat Scope 41
3.9 Testing Environment and Host Controls 42
3.9.1 Determinism and Reproducibility 42
3.10 Evaluation Metrics 43
3.11 Chapter Summary 43
Chapter Four — System Design and Implementation 44
4.1 Introduction 45
4.2 Design Principles 45
4.3 High-Level Architecture 45
4.4 The IFileScanner Abstraction and Orchestration 47
4.4.1 Extending the Pipeline with a New Layer 49
4.5 Structural Layers (1–4) 49
4.5.1 ExtensionWhitelist (Layer 1) 49
4.5.2 MagicBytes (Layer 2) 49
4.5.3 HeaderContentMatching (Layer 3) 50
4.5.4 DoubleExtension (Layer 4) 50
4.5.5 Error Handling and Fail-Closed Behavior 50
4.6 Detection Layers (5, 5b, 5c) 51
4.6.1 YARA Signature Scanning (Layer 5) 51
4.6.2 PDF FlateDecode Recursive Inflation (Layer 5b) 51
4.6.3 Embedded-Link Inspection (Layer 5c) 52
4.7 Transformation Layers (6, 7) 52
4.7.1 Image Re-encoding (Layer 6) 53
4.7.2 SVG Sanitization (Layer 7) 54
4.8 Caching Subsystem 55
4.9 Configuration Model 56
4.10 Security-Headers Middleware 56
4.11 Verdicts and the Evidence Trail 57
4.11.1 Reading a Trace 57
4.12 The Instrument-Panel Frontend 57

4.14 Worked Example: A File's Journey Through the Pipeline 58
4.15 Chapter Summary 59
Chapter Five — Experimental Results 60
5.1 Introduction 61
5.2 Evaluation Setup 61
5.3 Aggregate Results 61
5.3.1 Confusion Matrix and Metrics 61
5.3.2 Interpretation 63
5.3.3 False-Positive Analysis 63
5.3.4 False-Negative Analysis 64
5.4 Adversarial-Corpus Comparison 64
5.5 Per-Sample Analysis 66
5.5.1 EICAR Behind a Single FlateDecode 66
5.5.2 EICAR Behind a Double FlateDecode 66
5.5.3 Appended-Payload Neutralization 67
5.5.4 Cache Behavior 67
5.5.5 PNG Text-Chunk Parasite 67
5.5.6 WebP with Appended Bytes 68
5.5.7 End-to-End Walkthrough of a Neutralized Upload 68
5.6 Neutralization of Pixel-Domain Steganography (LSB) 69
5.6.1 Dataset and Metric 69
5.6.2 Results 70
5.6.3 Technique and Scope 71
5.7 Timing and Performance 72
5.8 Contextual Comparison with Machine-Learning Detectors 73
5.9 Comparison with Related Systems 73
5.10 Chapter Summary 73
Chapter Six — Discussion 75
6.1 Introduction 76
6.2 Why Layering Resists Evasion 76
6.3 The Value of Ordering 76
6.4 Self-Bypass Findings and Fixes 76
6.4.1 Nested-Filter Evasion 77
6.4.2 The Antivirus / Scanner Race 77

6.4.3 Stale-Cache Masking 77
6.5 Detection versus Neutralization 78
6.6 Transparency and Auditability 78
6.7 Qualitative Comparison with Prior Approaches 78
6.8 Limitations 79
6.9 Threats to Validity 80
6.10 Deployment Considerations 80
6.11 Ethical Considerations 81
6.12 Chapter Summary 81
Chapter Seven — Conclusion and Future Work 82
7.1 Summary of the Work 83
7.2 Revisiting the Research Questions 83
7.3 Contributions 84
7.4 Immediate Next Steps 84
7.5 Longer-Term Directions 84
7.6 Reflections and Lessons Learned 85
7.7 Concluding Remarks 85
References 86
Appendices 88
Appendix A — Representative YARA Rules 89
Appendix B — Example Per-Layer Trace 89
Appendix C — Adversarial Corpus Composition 90
Appendix D — Configuration Example 91
Appendix E — Reference-Scanner Logic 91
Appendices 92
Appendix A — Representative YARA Rules 93
Appendix B — Example Per-Layer Traces 93
Appendix C — Adversarial Corpus Composition 94
Appendix D — Example Configuration 95
Appendix E — Reference-Scanner Logic 95
Appendix F — Adversarial Fixtures and Expected Verdicts 95
Appendix G — YARA Rule Inventory 96

List of Figures

A taxonomy of file-upload attacks 27
Anatomy of a polyglot file 29
Recursive inflation of a nested FlateDecode stream 31
Threat model and trust boundary 39
SecureUploader pipeline architecture 46
Pipeline orchestration via IFileScanner 47
CachedScanLayer three-part cache key 55
Confusion matrix and performance metrics 62
Aggregate detection metrics 62
False-positive rate before and after severity-gating 63
Detection / neutralization matrix 65
Adversarial-corpus detection rate by scanner 65
Neutralization by image re-encoding 67
Pixel-domain (LSB) neutralization before and after LSB clearing 71
Relative per-layer processing cost 72
Qualitative comparison vs prior approaches 79

List of Tables

The scope of this project: what is addressed and what is deliberately excluded. 17
Principal classes of file-upload attacks and the property each abuses. 27
Polyglot construction taxonomy (after Mitra) and the layer that defeats each class. 29
Mapping of each evasion technique to the pipeline layer(s) that defeat it. 34
Comparison of the core file-inspection strategies used in the pipeline. 34
Development platform, language, and key libraries. 38
Threat-model assumptions: attacker capabilities, goals, and what is out of scope. 39
Evaluation datasets and their roles in the corpus. 39
Accepted file types and their in-scope threat classes. 41
Definitions of the evaluation metrics used in Chapter Five. 43
The nine inspection layers, their category, and their responsibility. 46
Categories of YARA rules in the signature-scanning layer. 51
SVG constructs removed by the sanitization layer and the risk each poses. 54
Runtime configuration sections exposed by the pipeline. 56
The six reference scanners and the documented validation logic of each. 64
Per-fixture verdict of SecureUploader on representative adversarial samples. 66
The pixel-domain steganography dataset and the tested sample. 70
Per-class neutralization of LSB payloads on the tested sample. 71
Contextual comparison with machine-learning detectors on CIC-Evasive-PDFMal2022. 73
The three self-bypass vulnerabilities found during development and their fixes. 77
Threats to validity and the mitigation applied to each. 80

List of Abbreviations and Technical Terms

Abbreviation	Full Term
API	Application Programming Interface
AV	Antivirus
CIC	Canadian Institute for Cybersecurity
CWE	Common Weakness Enumeration
CVE	Common Vulnerabilities and Exposures
DoS	Denial of Service
DVWA	Damn Vulnerable Web Application
EICAR	European Institute for Computer Antivirus Research (test file)
FN	False Negative
FP	False Positive
FPR	False-Positive Rate
HTTP	Hypertext Transfer Protocol
MIME	Multipurpose Internet Mail Extensions
PDF	Portable Document Format
PE	Portable Executable
RCE	Remote Code Execution
RTP	Real-Time Protection
SHA-256	Secure Hash Algorithm, 256-bit
SSRF	Server-Side Request Forgery
SVG	Scalable Vector Graphics
TN	True Negative
TP	True Positive
XSS	Cross-Site Scripting
XXE	XML External Entity
YARA	Pattern-matching engine for malware classification

Chapter One — Introduction

1.1 Introduction

File upload is a near-universal feature of modern web applications. Content-management systems, e-learning portals, ticketing systems, medical and financial platforms, and social networks all expose at least one endpoint that accepts a file from an untrusted user and then stores, processes, or serves it. Each of these three actions turns a property of the uploaded bytes into a capability the attacker may be able to exploit, which is what makes file upload one of the largest and most consequential attack surfaces on the web.

Consider a concrete scenario. A photo-sharing site allows members to upload a profile picture, checking only that the filename ends in an image extension. An attacker uploads a file named avatar.php.jpg whose bytes begin with a valid image header but end with a small server-side script. If the server stores the file under a web-executable path and a later request reaches it as a script, the attacker has achieved remote code execution on a trusted host — from nothing more than a profile picture. Variations of this pattern, catalogued under CWE-434 (Unrestricted Upload of File with Dangerous Type), have appeared repeatedly in real systems for two decades.

The core difficulty is that a file is not what its name says it is. An attacker controls every byte of the uploaded content and every character of its filename, and can therefore craft an input that passes a naive check while behaving maliciously once stored. The simplest form is extension spoofing — renaming a payload to end in an allowed extension. More advanced techniques defeat deeper checks: polyglot files are simultaneously valid instances of two formats, so that the same bytes are read as a harmless image by one parser and as executable script by another; malicious documents conceal their payload behind several layers of stream compression so that a single-pass scanner never sees the decoded content; and vector formats such as SVG carry active script that executes in the victim's browser when the file is served back.

Conventional upload validation applies a single check in isolation — an extension whitelist, a MIME-type sniff, or one signature scan — and treats a pass on that check as sufficient. Because each check reasons about only one property of the file, any input engineered to satisfy that one property is accepted regardless of what else it contains. The consequence is a long history of documented bypasses in which files that are individually accepted by every deployed check nonetheless carry an executable payload.

This project addresses that gap with SecureUploader, a defense-in-depth pipeline that inspects an uploaded file through nine ordered layers, each reasoning about a different property, so that defeating the pipeline requires defeating all applicable layers at once rather than any single one. Where a file cannot be cleanly accepted or rejected on structural grounds alone, SecureUploader introduces a second option — neutralization — that deterministically rewrites the file to remove any embedded payload while preserving its legitimate content, converting an ambiguous input into a demonstrably safe one.

1.2 Problem Statement

The problem this project confronts can be stated precisely. Given an uploaded file consisting of attacker-controlled bytes and an attacker-controlled filename, decide from the bytes alone whether the file is safe to store and serve, and do so in a way that cannot be defeated by an input engineered to satisfy any single check. A validator that reasons about one property — extension, MIME type, or a single signature scan — fails this requirement by construction, because the attacker can satisfy that one property while concealing a payload elsewhere in the file. What is required instead is a validator whose acceptance depends on multiple independent properties simultaneously, together with a mechanism for handling files that are neither clearly safe nor clearly malicious without simply discarding legitimate uploads.

1.3 Research Objective

The objective of this project is to design, implement, and empirically evaluate a file-upload security pipeline that resists the evasion techniques which defeat single-check validators, and to establish neutralization as a measurable outcome alongside detection. Concretely, the project pursues the following goals:

- **Layered detection.** Build a pipeline of independent inspection layers — extension whitelisting, magic-byte validation, header–content matching, double-extension detection, YARA signature scanning, recursive PDF stream inflation, embedded-link inspection, image re-encoding, and SVG sanitization — that must all be satisfied for a file to be accepted.
- **Evasion resistance.** Explicitly harden the pipeline against nested-filter PDF compression, image-carried scripts, and active SVG content, and validate that hardening on crafted adversarial fixtures rather than on benign or randomly-sampled malicious files alone.

- **Neutralization as a second metric.** Treat the deterministic rewriting of a file to strip a payload while preserving legitimate content as a first-class verdict, and measure it alongside detection so that value invisible to a detection-only account is captured.
- **Transparency.** Attach to every verdict a reproducible, per-layer evidence trail identifying exactly which layer acted and why, in contrast to opaque machine-learning detectors whose decisions cannot be audited.
- **Empirical evaluation.** Quantify detection performance on a large public corpus and compare the pipeline, layer by layer, against a set of reference validators on a hand-built adversarial corpus.

1.4 Research Questions

The objectives above translate into five research questions that structure the evaluation in the later chapters:

- **RQ1.** Can an ordered pipeline of independent layers resist evasion techniques — polyglots, nested-filter concealment, and active SVG content — that defeat single-check validators?
- **RQ2.** How does the pipeline perform, in standard detection metrics, on a large and realistic corpus of malicious and benign documents?
- **RQ3.** How does the pipeline compare, fixture by fixture, against representative reference validators on inputs constructed specifically to evade validation?
- **RQ4.** What measurable value does neutralization add beyond binary accept/reject detection, and what are its limits?
- **RQ5.** Does designing each layer with its own bypass in mind reveal weaknesses in the pipeline itself, and can those weaknesses be fixed?

1.5 Scope and Assumptions

The project is deliberately scoped to keep the threat tractable and the evaluation meaningful. The pipeline accepts a narrow set of image and document types and inspects raw bytes without any learned model; formats whose semantics broaden the attack surface, and defenses that require training or full dynamic sandboxing, are out of scope. The boundaries are summarized below and revisited when the threat model is defined in Chapter Three.

.

In scope	Out of scope
Raster (JPEG, PNG, WebP) and vector (SVG) images, and PDF documents	Office documents, archives, and executable formats
Byte-level inspection with no training phase	Machine-learning training and feature extraction
Neutralization by re-encoding and sanitization	Full dynamic sandboxing / detonation
Single-file uploads at a public endpoint	Recursive expansion of container/archive uploads

Table 1: The scope of this project: what is addressed and what is deliberately excluded

Two assumptions bound the work. First, the analysis host and the pipeline itself are trusted; the attacker acts only through the upload endpoint. Second, the accepted file types are almost entirely non-executable on the host, which is what makes controlled testing with real malicious samples defensible under the host controls described in Chapter Three.

1.6 Practical Applications

The pipeline is designed as a reusable component that can be embedded wherever untrusted files enter a system. Its practical applications include the following.

1.6.1 Hardening Web-Application Upload Endpoints

SecureUploader can sit directly in front of any upload handler, rejecting or neutralizing malicious files before they are stored or served. Because it reasons about file content rather than trusting the filename or the client-supplied content type, it protects endpoints that would otherwise be bypassed by extension spoofing, MIME confusion, or polyglot inputs. Its narrow accepted-type set makes it a natural fit for the common case of an application that only ever needs to accept user images and documents.

1.6.2 Supporting Security Operations and Triage

The per-layer evidence trail produced for every file gives a security analyst an immediate, human-readable explanation of why a given upload was blocked or rewritten — which layer acted, on what evidence, and how long each stage took. This shortens triage, provides defensible documentation for each decision, and makes it possible to reproduce a verdict exactly rather than trusting an opaque score.

1.6.3 Integration into Continuous-Delivery Pipelines

Exposed as a service with a machine-readable verdict, the pipeline can gate artifact and asset uploads inside automated build and deployment workflows, preventing evasive files from entering a trusted software supply chain. Because it is training-free and deterministic, the same input always yields the same verdict, which is a prerequisite for use inside a reproducible build.

1.6.4 A Foundation for Research and Coursework

Because each layer is independent and its contribution is measurable in isolation, the pipeline serves as a practical platform for studying evasion techniques, ablating individual defenses, and exploring the trade-off between detection and neutralization. Its transparent design makes it suitable as a teaching artifact in a security curriculum.

1.7 Challenges

Building an upload pipeline that is both safe and usable raises several tensions that shaped the design of SecureUploader:

- **Coverage versus false positives.** A stricter policy blocks more attacks but also rejects more legitimate files. The pipeline must maximize detection while keeping the false-positive rate on genuine files low enough to be deployable, and must be able to explain and tune the causes of any false rejection.
- **Evasion by construction.** Adversaries craft inputs specifically to satisfy each known check, so every layer must be designed with its own bypass in mind and validated against inputs built to defeat it, not merely against representative samples.
- **Nested and recursive concealment.** Payloads hidden behind several layers of compression are invisible to a single-pass scanner; inflation must be recursive and bounded, expanding concealed content to a configurable depth without exposing the pipeline to decompression-bomb denial of service.
- **Neutralization without data loss.** Rewriting a file to strip a payload must be deterministic and must preserve the file's legitimate, visible content, so that a neutralized file remains useful to the end user rather than being silently corrupted.
- **Interaction with the host environment.** Security tooling on the analysis host can interfere with detection — for example by quarantining a sample before the pipeline can inspect it —

so the evaluation environment itself must be controlled, documented, and reasoned about as part of the methodology.

1.8 Literature Review

This section surveys the prior work most directly related to SecureUploader: published upload-checking systems, evasion taxonomies, empirical studies of polyglot files, machine-learning detectors for malicious documents, and the re-encoding defenses on which the neutralization layers build. Each body of work is examined for what it contributes and where it stops, and the section closes by stating the research gap that SecureUploader addresses.

1.8.1 Published Upload-Checking Systems (FileUploadChecker)

The closest published analog to this work is FileUploadChecker (ARES 2022), a server-side component that inspects uploaded files against a set of format-specific rules before they are accepted. It validates that a file conforms to the structure its type implies and rejects files that fail those checks, and it is the most direct point of comparison for SecureUploader.

Two differences in framing separate the present work from FileUploadChecker. First, SecureUploader adds explicit evasion-resistance hardening for nested-filter and polyglot inputs — recursive inflation and re-encoding — rather than validating format conformance alone. Second, it introduces neutralization as an accepted outcome, so that an ambiguous but potentially legitimate file can be rewritten into an inert form instead of being rejected outright. FileUploadChecker is therefore treated as the primary baseline against which the contribution is positioned.

1.8.2 Evasion Taxonomies for File Upload (FUEL / Fuxploider-NG)

FUEL (DIMVA 2024) provides a systematic taxonomy of file-upload evasion scenarios together with an automated tool for exercising them against a target. Its value is descriptive: it enumerates the ways an upload check can be bypassed and organizes them into scenario categories.

In this project FUEL is used as a taxonomic reference rather than re-implemented as a comparator. Its scenario categories anchor the coverage map presented in Chapter Two, which states, for each evasion class, the pipeline layer intended to defeat it. Using FUEL this way keeps the evaluation grounded in an external taxonomy rather than one invented for the occasion.

1.8.3 Empirical Study of Polyglot Files (Koch et al.)

Koch et al. (ACM WWW 2025) present a large-scale study of the abuse and detection of polyglot files, including a catalog of real-world polyglot samples together with their cryptographic hashes. The study documents how frequently polyglots occur in the wild and how poorly common tools detect them.

That catalog supplies the real polyglot samples used in this project's adversarial corpus, retrieved by hash through a single sanctioned route. It also provides external support for the neutralization framing and for the rationale behind the image re-encoding layer, which destroys the secondary interpretation of a polyglot by rebuilding the file from decoded pixels — a defense the study's findings motivate directly.

1.8.4 Machine-Learning Detection of Malicious Documents

A substantial body of work applies machine learning to malicious-document detection. Representative studies on the CIC-Evasive-PDFMal2022 dataset report accuracies above 98% using classifiers trained on pre-extracted feature vectors — for example structural and metadata features of PDF documents fed to ensemble or stacking learners.

These results are strong but are not directly comparable to SecureUploader. They operate on curated feature tables with dedicated training and test splits, whereas SecureUploader operates on raw file bytes with no training phase and produces an auditable per-layer decision rather than a learned score. The comparison in this report is therefore contextual — situating a transparent, training-free, byte-level pipeline against opaque, trained, feature-level classifiers — rather than a head-to-head benchmark on the same operating regime.

1.8.5 Re-encoding and Round-Trip Sanitization

Prior work on image re-encoding as a sanitization defense (ICDR 2023) demonstrates that decoding an image to its pixel representation and re-encoding it destroys most appended or embedded payloads, because only the pixel data survives the round-trip. The same work characterizes the limits of the technique, notably that some steganographic payloads encoded into the pixel values themselves can survive certain round-trips.

These findings directly inform the design of the image re-encoding layer in SecureUploader and, equally importantly, the honest disclosure of its limitations: neutralization by re-encoding reduces

rather than wholly eliminates risk for the narrow class of pixel-level steganographic payloads, and this report states that limit rather than overclaiming.

1.8.6 The Research Gap

The surveyed work leaves a clear gap. Published upload checkers validate format conformance but do not systematically harden against nested-filter and polyglot evasion, nor do they treat neutralization as a measurable outcome. Evasion taxonomies describe attacks but do not provide a hardened, evaluated defense. Machine-learning detectors achieve high accuracy on curated feature sets but require training, operate as opaque classifiers, and do not act on raw bytes at the point of upload. Re-encoding studies establish the sanitization primitive but not an integrated pipeline around it. SecureUploader addresses this gap by combining an integrated, layered, byte-level pipeline with explicit evasion-resistance hardening, a second neutralization metric alongside detection, a transparent per-layer evidence trail, and an empirical evaluation on both a large public corpus and a hand-built adversarial corpus.

1.9 Structure of the Report

The remainder of this report is organized as follows. Chapter Two establishes the theoretical background: the file-upload attack surface, the evasion techniques that defeat naive validation, and the inspection primitives the pipeline composes. Chapter Three documents the methodology and environment, including the platform, the threat model, the evaluation datasets, the host controls, and the metrics used. Chapter Four presents the design and implementation of the nine-layer pipeline. Chapter Five reports the experimental results on both the public and the adversarial corpora. Chapter Six discusses why the design works, the self-bypass findings, the limitations, and the threats to validity. Chapter Seven concludes and outlines future work, and the References follow.

Chapter Two — Theoretical Background

2.1 Introduction

This chapter establishes the technical background needed to understand SecureUploader. It begins with how file formats are structured and how parsers read them, because the disagreements between parsers are the root cause of the most dangerous evasions. It then describes the file-upload attack surface, catalogs the principal classes of attack, and examines the three evasion techniques the pipeline is chiefly designed to defeat — polyglots, nested-filter concealment, and active SVG content. Finally it introduces the inspection building blocks the pipeline composes and the neutralization primitive that distinguishes it, closing with a coverage map and a comparison that motivate the layered design of Chapter Four.

2.2 File Formats and How Parsers Read Them

A file format is a convention for arranging bytes so that a program can recover structured meaning from them. Most formats begin with a magic number — a short, fixed byte sequence that identifies the type, such as FF D8 FF for JPEG, 89 50 4E 47 for PNG, or the literal characters that open a PDF. A parser typically reads this magic number, then follows the format's internal structure of headers, length fields, offsets, and trailing markers to locate the meaningful content.

Crucially, parsers do not all agree on where a file begins and ends. Some read strictly from the leading magic number and stop at a declared length; others scan for a trailing signature and read backwards; still others tolerate leading or trailing junk. This disagreement is not a bug in any one parser but an emergent property of many independent implementations, and it is exactly what an attacker exploits: a single byte sequence can satisfy one parser as a complete, valid image while a second parser, using a different rule, finds and acts on a payload the first ignored.

Two further properties of real formats matter here. First, many formats contain comment or metadata regions whose contents are ignored by the rendering path, giving an attacker a place to hide bytes that survive naive validation. Second, container formats such as PDF allow content to be encoded by a chain of filters, so that the meaningful bytes are not present literally in the file but must be reconstructed by applying each filter in turn. Both properties are leveraged by the evasions described later in this chapter.

2.2.1 JPEG Structure

A JPEG file is a sequence of segments, each introduced by a two-byte marker. It opens with the start-of-image marker (FF D8) and closes with the end-of-image marker (FF D9), and in between carries application and comment segments that hold metadata, followed by the compressed image data. The comment segment is significant for security because its contents are ignored by the rendering path: an attacker can place arbitrary bytes there, and a naive validator that checks only the opening marker will accept the file. Equally important, many JPEG readers stop at the end-of-image marker and ignore anything after it, so bytes appended past FF D9 are invisible to the image while remaining available to a second parser — the basis of the appended-payload and zipper polyglots this project neutralizes by re-encoding.

2.2.2 PNG Structure

A PNG file begins with an eight-byte signature and is then organized as a series of length-prefixed chunks, each carrying a four-character type code and a checksum. Some chunk types are critical to rendering; others, such as textual-metadata chunks, are ancillary and are ignored by the image itself. As with the JPEG comment segment, these ancillary chunks are a natural hiding place for a payload that survives a superficial check, which is why the parasite construction targets them and why rebuilding the image from decoded pixels — which keeps only the critical pixel data — removes them.

2.2.3 PDF Object Model

A PDF is a collection of numbered objects — dictionaries, arrays, and streams — linked by a cross-reference table and organized under a document catalog. Streams may carry a chain of filters that must be applied in order to recover their content, which is the mechanism behind nested-filter concealment. Objects may also declare actions, including automatic actions that fire when the document is opened and actions that launch or fetch external resources, which is the mechanism behind malicious embedded links. This dual structure — filtered streams plus actionable objects — is why the pipeline dedicates a recursive-inflation layer and an embedded-link layer specifically to PDFs, rather than treating them as opaque blobs.

2.2.4 SVG and the XML Model

An SVG file is an XML document whose elements describe vector shapes. Because it is XML, it inherits the full XML feature set, including document-type declarations that can define external

entities, and because it is a web-native image format, browsers execute the script and event handlers it contains when it is rendered. An SVG is therefore simultaneously a picture and a potential program, and unlike a raster image it cannot be reduced to a grid of pixels without losing its meaning. This is why SVG is neutralized not by re-encoding but by parsing the XML and removing the specific dangerous constructs while preserving the drawing, a fundamentally different sanitization strategy from the one used for raster images.

2.3 The File-Upload Attack Surface

An upload endpoint accepts attacker-controlled bytes and typically performs one or more of three dangerous actions with them, each of which turns a property of the file into a capability for the attacker.

- **Storage.** A file written to a path that may later be served or executed becomes dangerous if its bytes are, or can be interpreted as, code. A script stored under a web-executable directory and reachable by URL yields remote code execution.
- **Processing.** A file handed to a parser — an image library, a PDF reader, an XML processor — can trigger memory corruption, entity expansion, or external-resource fetches if the parser is vulnerable or misconfigured, turning a parse into code execution, denial of service, or server-side request forgery.
- **Serving.** A file returned to other users with an active content type executes in their browsers. An SVG or HTML payload served from a trusted origin yields stored cross-site scripting against every viewer.

The essential asymmetry across all three is that the server must decide, from the bytes alone, whether the file is safe, while the attacker is free to construct bytes that satisfy whatever test the server applies. The attack classes below all exploit this asymmetry, and the pipeline in Chapter Four is organized to remove the asymmetry one property at a time.

2.3.1 HTTP Upload Mechanics

A browser typically submits a file using a multipart/form-data request, in which each part carries a small header declaring a field name, a filename, and a content type, followed by the raw bytes of the file. Every one of those declared values is supplied by the client and is therefore attacker-controlled: the filename can be any string, and the content type can be set to any value regardless

of what the bytes actually are. A server that keys its acceptance decision on the declared filename or the declared content type is trusting the attacker to describe the attack honestly, which is why reliable validation must derive the file's type from its bytes rather than from the request metadata.

This distinction between declared and actual type is the seam that several attacks in this chapter exploit. Extension spoofing forges the filename; MIME spoofing forges the content type; and the more advanced polyglot and nested-payload attacks forge the leading bytes themselves so that even a byte-level type check is satisfied. The pipeline therefore treats the request metadata as a hint to be verified, never as a fact to be trusted, and escalates from cheap metadata checks to progressively deeper byte-level inspection.

2.3.2 Storage, Processing, and Serving Paths

What the server does with an accepted file determines which properties become dangerous. If the file is written under a directory from which the web server will execute scripts, then any bytes that can be interpreted as server-side code become a remote-code-execution vector, and the safe practice is to store uploads outside any executable path under server-generated names. If the file is handed to a parser, the parser's own vulnerabilities become reachable by the attacker, so the pipeline prefers to rebuild files rather than merely parse them where possible. And if the file is later served back to users, the content type under which it is served governs whether active content executes, which is why the security-headers middleware described in Chapter Four constrains that serving policy.

The attacker-controlled filename is itself a hazard beyond its extension. A filename containing path-traversal sequences can, on a careless server, cause the file to be written outside its intended directory, and a filename that collides with an existing file can overwrite it. Deriving the stored name from server-side state rather than from the uploaded filename removes this class of problem, and treating the filename purely as an untrusted label — inspected but never used to locate storage — is part of the fail-closed posture the pipeline adopts.

2.4 A Taxonomy of File-Upload Attacks

The principal classes of file-upload attack are summarized in Table 2 and organized as a taxonomy in Figure 1, then examined individually.

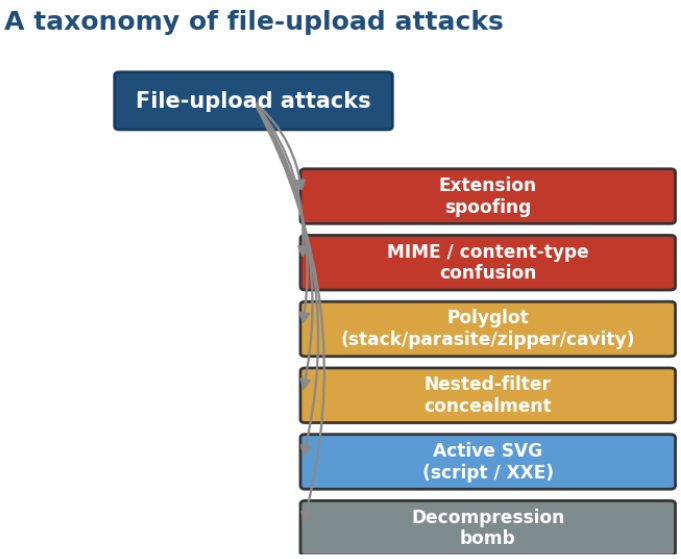


Figure 1: A taxonomy of file-upload attacks

Attack class	Property abused	Typical impact
Extension spoofing	Trust in the filename extension	Web-shell upload, RCE
MIME / content-type spoofing	Trust in the client-supplied MIME type	Bypass of type filters
Polyglot upload	A file valid in two formats at once	Stored XSS, RCE via second parser
Compressed / nested payload	Payload hidden behind stream filters	Signature-scanner evasion
Active SVG content	Script and external entities in SVG	Stored XSS, XXE, SSRF
Decompression bomb	Extreme expansion ratio	Denial of service

Table 2: Principal classes of file-upload attacks and the property each abuses.

2.4.1 Extension Spoofing

The cheapest evasion renames a payload to end in an allowed extension — shell.php becomes shell.jpg, or the double form shell.php.jpg exploits servers that key execution off an interior extension. It defeats any validator that trusts the filename and is neutralized only by inspecting the actual bytes.

2.4.2 MIME / Content-Type Spoofing

The HTTP request that carries an upload includes a client-supplied content-type header. A validator that trusts this header is bypassed by simply setting it to an allowed value, because the

header is attacker-controlled metadata rather than a property of the bytes. Reliable typing requires deriving the type from the content itself.

2.4.3 Web Shells

A web shell is a small server-side script that, once stored under an executable path and requested, gives the attacker a command interface on the host. Web shells are the classic end goal of extension spoofing and image-carried-script polyglots, and they are why storage-path handling and content typing are treated as security-critical rather than cosmetic.

2.4.4 Polyglot Uploads

A polyglot file is simultaneously a valid instance of two or more formats, so that a check for one format passes while a second parser downstream acts on a concealed payload. Polyglots are examined in depth in the next section because they are the central evasion this project targets.

2.4.5 Stored Cross-Site Scripting via Uploads

When an uploaded file is served back with an active content type — most commonly an SVG containing script — the script runs in the browser of every user who views it, from the trusted origin of the hosting site. This turns a passive upload into stored cross-site scripting against the whole user base.

2.4.6 XXE and SSRF via XML-Based Formats

SVG is XML, and XML permits external-entity declarations. A malicious SVG can declare an entity that references a local file or an internal URL; when the document is parsed, the entity is resolved, disclosing file contents (XXE) or causing the server to issue attacker-directed requests (SSRF). The defense is to parse the document and remove the dangerous declarations before it is trusted.

2.4.7 Compression and Decompression Attacks

Two distinct attacks exploit compression. In the first, a payload is hidden behind one or more stream filters so that a single-pass scanner never sees it. In the second, a small file is crafted to expand enormously when decompressed, exhausting memory or disk — a decompression bomb. A robust pipeline must inflate nested content to expose hidden payloads while bounding the work it performs so that it cannot itself be turned into a denial-of-service vector.

2.5 Polyglot Files in Depth

Because format parsers disagree about where a file begins and ends, a single byte sequence can be a valid image to one parser and a valid script or archive to another. Figure 2 illustrates the essential idea: the same bytes on disk are read by an image parser as a legitimate image and by a script engine or archiver as an executable payload.

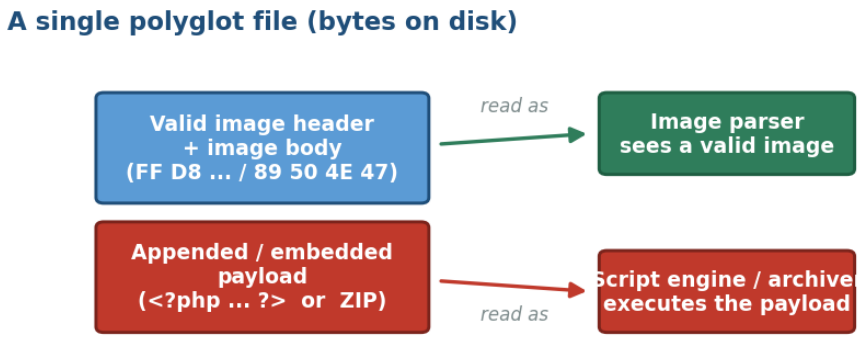


Figure 2: Anatomy of a polyglot file

Polyglots are constructed in a small number of recurring ways. Following the taxonomy popularized by the Mitra tool, the principal constructions are stack, parasite, zipper, and cavity. Each hides the second format in a different structural niche, and each is defeated by a specific pipeline layer, as summarized in Table 3.

Construction	How it works	Defeated by
Stack	Two formats concatenated; parsers start at opposite ends	Header–content matching, re-encoding
Parasite	Second format hidden in a comment / metadata chunk	Image re-encoding (drops chunks)
Zipper	Archive appended after a valid image	Re-encoding, embedded inspection
Cavity	Payload placed in unused / padding bytes	Re-encoding (rebuild from pixels)

Table 3: Polyglot construction taxonomy (after Mitra) and the layer that defeats each class.

The common thread is that in every construction the malicious second format is not part of the image's pixel data. This is the key insight the pipeline exploits: rebuilding the image from its

decoded pixels discards everything that is not a pixel — the appended archive, the parasite chunk, the cavity payload — and so neutralizes the polyglot regardless of which construction produced it.

2.5.1 A Worked Polyglot Example

To make the mechanism concrete, consider a zipper polyglot built by appending a ZIP archive to the end of a valid JPEG. The file begins with the JPEG start-of-image marker and contains a complete, well-formed JPEG that any image viewer renders correctly; after the JPEG's end-of-image marker, the attacker appends the bytes of a ZIP archive. Because a ZIP archive is located by scanning backwards from the end of the file to its central directory, an archive tool reading the same bytes finds and extracts the ZIP, while the image viewer, reading forwards from the start, stops at the end-of-image marker and never notices the appendage. The one file is thus simultaneously a valid image and a valid archive.

This example also shows why no single cheap check suffices. The extension is a legitimate image extension, so whitelisting passes; the leading bytes are a genuine JPEG header, so magic-byte validation passes; and the image renders, so an image-size probe passes. Only two properties betray the file: the bytes after the end-of-image marker make the declared and actual content lengths disagree, which header–content matching can detect, and rebuilding the image from pixels discards the appended archive entirely, which is what the re-encoding layer does. The same file that defeats three metadata checks is therefore neutralized by the structural and transformation layers acting on properties the attacker could not simultaneously forge.

2.6 Nested-Filter and Compression Concealment

Document formats such as PDF allow a stream to be encoded by a chain of filters — for example several successive FlateDecode passes. A scanner that decodes only the outermost layer, or that pattern-matches the raw stored bytes, never observes the decoded payload. Figure 3 shows the effect: the payload is reached only after the nested filters are inflated in turn.

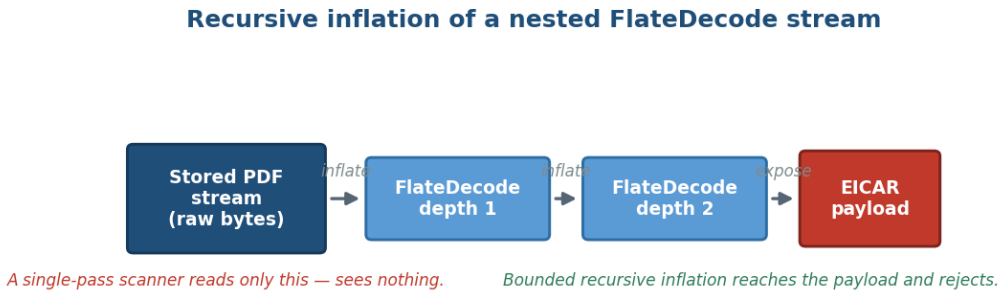


Figure 3: Recursive inflation of a nested FlateDecode stream

Defeating this evasion requires recursively inflating nested filters to a bounded depth before scanning, while guarding against decompression bombs that expand to exhaust memory. The bound on depth and on expanded size is what separates a safe implementation from one that can itself be attacked; Chapter Four describes how the pipeline sets and enforces those bounds.

2.6.1 Decompression Bombs and Expansion Ratios

Recursive inflation introduces a hazard of its own. A decompression bomb is a small input crafted to expand to an enormous size when decompressed — a few kilobytes on disk becoming gigabytes in memory — by exploiting the very high compression ratios achievable on highly redundant data. A pipeline that inflates nested streams without limit can be turned by such an input into a denial-of-service vector against itself, exhausting memory or disk in the course of trying to inspect the file.

The defense is to bound two quantities independently: the depth of recursion, so that an attacker cannot force an unbounded number of inflation passes, and the total expanded size, so that any single pass that produces more than a configured maximum is abandoned rather than completed. Enforcing both bounds converts inflation from a liability into a safe operation: a legitimate nested document inflates within the limits and is scanned, while a bomb trips a limit and is rejected before it can do harm. The specific limits are exposed as configuration, so a deployment can trade inspection depth against resource exposure according to its own risk tolerance.

2.7 Active Content in Vector Formats

SVG is an XML dialect that may contain script elements, event-handler attributes such as onload, foreignObject nodes embedding arbitrary HTML, and external-entity declarations. When an SVG is served back to users, this active content executes in their browsers, yielding stored cross-site scripting; external entities additionally enable file disclosure and server-side request forgery. Because the danger lies in specific elements and attributes rather than in the file's overall structure, the defense is not to reject SVGs but to parse them and strip the dangerous constructs while preserving the vector drawing — a form of neutralization particular to XML-based formats.

2.8 Inspection Building Blocks

The pipeline is assembled from a small set of inspection primitives, each of which reasons about a different property of the file. None is sufficient alone; their complementary strengths are what make composition worthwhile.

2.8.1 Extension Whitelisting

An allow-list of permitted extensions is the first and cheapest gate. On its own it is trivially bypassed by spoofing, but as the first layer of a pipeline it eliminates the large class of obviously-disallowed types and narrows the set of parsers that later, more expensive layers must consider.

2.8.2 Magic-Byte / Signature Validation

Every well-formed file of a given type begins with a characteristic magic number. Validating that the leading bytes match the claimed extension defeats naive extension spoofing, because a renamed script does not begin with a valid image header. It does not, however, defeat polyglots, whose leading bytes are a genuine image header by construction.

2.8.3 Header–Content Matching

Header–content matching goes further than the magic number, checking that the structure implied by the header is consistent with the rest of the file — that declared lengths, offsets, and trailing markers are coherent. The inconsistencies introduced by stacking or appending a second format, which a magic-byte check misses, are surfaced here.

2.8.4 YARA Signature Scanning

YARA is a rule-based pattern-matching engine widely used to classify files by textual and binary signatures. A curated rule set can flag known-malicious markers — embedded scripts, shell tokens,

document actions, or the EICAR test string — anywhere in the content it is given. Signature scanning is precise on known patterns but blind to payloads it has no rule for, which is why it is one layer among several rather than the whole defense, and why it must be fed decoded rather than raw bytes when payloads are concealed by compression.

2.8.5 Content Re-encoding and Sanitization

The final building block does not merely judge a file; it rewrites it. Decoding an image to its raw pixels and re-encoding it from scratch discards every byte that is not part of the pixel data — appended archives, parasite chunks, and cavity payloads all vanish — while the visible image is preserved. Parsing an SVG and removing script, event handlers, foreignObject nodes, and external entities likewise preserves the drawing while eliminating its active content. This is the mechanism behind neutralization: an ambiguous but potentially useful file is transformed into a provably inert one rather than being rejected.

2.9 Neutralization as a Distinct Outcome

Detection answers a yes/no question: is this file malicious? Neutralization answers a different one: can this file be made safe without discarding its legitimate content? The two are complementary. A file that is clearly malicious should be rejected; a file that is clearly benign should be accepted; but a large middle class of files — an image with appended bytes, an SVG with a stray handler — is neither, and for that class neutralization offers a third outcome that neither rejects a possibly-legitimate upload nor admits a possibly-dangerous one. Treating neutralization as a first-class, measurable outcome rather than a side effect is one of the distinguishing choices of this project, and its limits — chiefly the narrow class of pixel-level steganographic payloads that can survive a re-encode — are stated honestly rather than hidden.

2.10 Mapping Evasion Techniques to Defenses

Anchoring the design in an explicit coverage map makes it possible to state, for each evasion technique, exactly which layer is responsible for defeating it. Table 4 gives that mapping; it is the contract that Chapter Five's adversarial evaluation tests fixture by fixture.

Evasion technique	Defending layer(s)
Extension spoofing	ExtensionWhitelist (1), MagicBytes (2)
MIME / content-type spoofing	MagicBytes (2), HeaderContentMatching (3)
Double / disguised extension	DoubleExtension (4)
Image polyglot (stack / parasite / cavity)	ImageReEncoding (6)
Zipper (appended archive)	HeaderContentMatching (3), ImageReEncoding (6)
Known malicious signature	SignatureScanning (5)
Nested-filter PDF payload	PdfFlateDecode (5b)
Embedded malicious link / action	EmbeddedLink (5c)
Active SVG content (script, XXE)	SvgSanitization (7)
Decompression bomb	Bomb guards in (5b) and (6)

Table 4: Mapping of each evasion technique to the pipeline layer(s) that defeat it.

2.11 Comparison of Detection Strategies

No single strategy is sufficient. Cheap metadata checks are easily forged; signature scanning is blind to novel payloads; re-encoding is powerful but applies only to formats that can be safely rebuilt. Their complementary strengths and weaknesses, summarized in Table 5, are precisely what motivates composing them into an ordered pipeline in which each layer covers the blind spot of another.

Strategy	Strength	Weakness
Extension whitelist	Cheap first gate	Trivially spoofed alone
Magic-byte validation	Defeats naive spoofing	Passes polyglots
Header–content matching	Surfaces structural inconsistency	Misses semantic payloads
YARA signature scan	Precise on known patterns	Blind to unknown payloads
Recursive inflation	Exposes nested payloads	Cost; bomb risk if unbounded
Re-encoding / sanitization	Neutralizes rather than only detects	Format-specific; rare stego survival

Table 5: Comparison of the core file-inspection strategies used in the pipeline.

2.12 Chapter Summary

File upload is dangerous because the server must judge attacker-controlled bytes, and every cheap check corresponds to a property the attacker can forge. Parser disagreement makes polyglots possible; stream filters make nested concealment possible; XML makes active SVG content possible. The inspection building blocks each cover part of the threat and each have blind spots, which is why the next chapters compose them, in a deliberate order, into a layered pipeline with re-encoding-based neutralization as a distinct outcome, guided by the coverage map of Table 4.

Chapter Three — Methodology and Environment

3.1 Introduction

This chapter documents the methodology and the engineering environment in which SecureUploader was built and evaluated: the research approach, the development platform and libraries, the threat model that bounds what the pipeline is responsible for, the datasets that make up the evaluation corpus, the controlled host environment used for testing with real malicious samples, and the metrics by which results are reported in Chapter Five.

3.2 Research Methodology

The project follows an engineering, design-science methodology: an artifact — the pipeline — is designed to solve a stated problem, built iteratively, and evaluated against explicit criteria. Development proceeded layer by layer, with each layer verified against fixtures that exercise its specific responsibility before the next was added, so that the contribution of each layer could be attributed and, later, ablated. Evaluation was designed up front around two complementary corpora — a large public corpus for aggregate metrics and a hand-built adversarial corpus for evasion resistance — so that neither realistic-but-unengineered files nor engineered-but-unrealistic ones would be relied on alone. The self-bypass mindset, in which each layer is attacked as if by an adversary who knows it exists, was applied throughout and is itself reported as a result in Chapter Six.

3.3 Development Platform and Language

SecureUploader is implemented in C# on .NET 8 using ASP.NET Core. A managed, strongly-typed platform was chosen for three reasons. First, memory safety removes an entire class of vulnerabilities that would otherwise be a concern when parsing untrusted files, so that a malformed input cannot corrupt the pipeline itself. Second, the ASP.NET Core middleware model maps naturally onto an ordered pipeline of inspection layers, each implementing a common interface and each able to short-circuit the request. Third, the ecosystem provides mature, maintained libraries for image processing and pattern matching, avoiding the need to hand-roll parsers for hostile input — which would reintroduce exactly the parsing vulnerabilities the project seeks to avoid.

3.4 Libraries, Tools, and Environment

The implementation relies on a small, deliberately-chosen set of libraries and tools. Image decoding and re-encoding are performed with Magick.NET, upgraded off a known-vulnerable release so that the sanitization primitive is not itself a liability; signature scanning uses a YARA rule set of twenty rules including an EICAR rule; and fixture generation for the adversarial corpus uses the open-source Mitra tool. The development and evaluation environment is summarized in Table 6.

Component	Choice
Language / runtime	C# on .NET 8
Web framework	ASP.NET Core (middleware pipeline)
Image processing	Magick.NET (patched release)
Signature engine	YARA (20 rules, incl. EICAR)
Fixture generation	Mitra (Ange Albertini)
Frontend	React instrument-panel UI
Host OS (testing)	Windows, hardened lab configuration

Table 6: Development platform, language, and key libraries.

3.5 Threat Model

A defense is only meaningful relative to an explicit threat model. In this project the attacker controls every byte of the uploaded file and every character of its filename, and acts solely through the public upload endpoint, as depicted in Figure 4. The attacker's goals include remote code execution, stored cross-site scripting, XXE and SSRF, denial of service, and the hosting of malware on the trusted origin. Out of scope are attacks that do not pass through the endpoint — network interception, compromise of the host operating system, or an insider with direct server access — and the pipeline and its host are assumed trusted, with the one documented exception that host antivirus can interfere with measurement, which is addressed in the host controls.

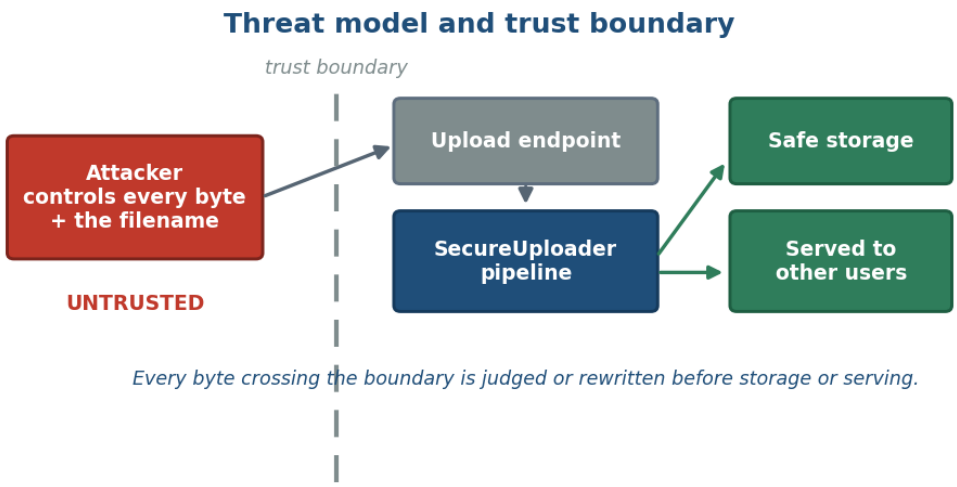


Figure 4: Threat model and trust boundary

Aspect	Assumption
Attacker control	Full control of file bytes and filename
Attacker goals	RCE, stored XSS, XXE / SSRF, DoS, malware hosting
Delivery channel	The public upload endpoint only
Trusted components	The pipeline and the analysis host
Out of scope	Network interception, host OS compromise, insider access

Table 7: Threat-model assumptions: attacker capabilities, goals, and what is out of scope.

3.6 Evaluation Datasets and Corpus Construction

Evaluation uses two complementary corpora. A large public corpus supplies aggregate detection statistics on realistic malicious and benign documents, and a smaller hand-built adversarial corpus stress-tests the pipeline against inputs constructed specifically to evade it. The datasets and their roles are summarized in Table 8.

Dataset	Role in evaluation
CIC-Evasive-PDFMal2022	10,012-file aggregate corpus (5,549 malicious / 4,463 clean)
Koch et al. polyglot catalog	Real polyglot samples (by hash) for the adversarial corpus
MalwareBazaar (abuse.ch)	Sole download route for real malicious samples
Kaggle natural-images	Benign image set for false-positive testing
Inert fixtures (Mitra)	Crafted safe polyglots and evasion fixtures

Table 8: Evaluation datasets and their roles in the corpus.

3.6.1 The Public Corpus (CIC-Evasive-PDFMal2022)

The primary quantitative evaluation uses the full CIC-Evasive-PDFMal2022 corpus of 10,012 files, comprising 5,549 malicious and 4,463 clean PDF documents. The dataset is designed to include evasive malicious PDFs — documents built to slip past detectors — which makes it a demanding test of detection rather than a collection of trivially-flagged samples, and therefore a fair basis for the aggregate metrics reported in Chapter Five. It is worth stating a property of the benign half up front: the clean documents in this corpus were not sampled at random but selected to resemble the malicious class, so that byte-level and signature indicators are, by construction, weak discriminators on the clean set. This makes the corpus a deliberately adversarial basis for the false-positive analysis, and the false-positive results in Chapter Five should be read with that construction in mind rather than as an estimate over a random real-world upload population.

3.6.2 Real Polyglots and the Download Route

Real polyglot samples are drawn by hash from the Koch et al. catalog and retrieved exclusively through MalwareBazaar using a free authentication key. Restricting downloads to a single sanctioned route keeps the handling of live malware auditable. Some older hashes are unavailable through that route; these are logged as a data-availability limitation rather than silently dropped, so that the size of the real-sample portion of the corpus is reported honestly.

3.6.3 Benign Corpus and Inert Fixtures

Legitimate images from the Kaggle natural-images set provide the benign population for false-positive measurement, giving a realistic distribution of the images a real deployment would accept. A set of inert fixtures generated with Mitra provides crafted polyglots and evasion cases whose behavior is fully known in advance, which is what makes the adversarial evaluation interpretable: for each fixture the expected verdict is known, so a wrong verdict is unambiguous.

Corpus construction followed a few disciplines that keep the evaluation honest. Real samples are identified by their SHA-256 hash and retrieved by that hash, so there is no ambiguity about which sample a result refers to and no risk of silently substituting a different file. Samples are deduplicated by hash so that an identical file cannot be counted twice and inflate a rate. And for every category the count actually obtained is recorded rather than the count intended, so that when some hashes prove unavailable the reported corpus size reflects what was truly tested. These

practices matter because a detection rate is only as trustworthy as the inventory it is computed over.

The two corpora play deliberately different roles and are never blended. The public corpus provides a population-level view: a balanced set of realistic malicious and clean documents over which aggregate metrics are meaningful. The adversarial corpus provides a capability-level view: a set of engineered fixtures over which the question is not what fraction is caught in a realistic population but whether each specific evasion is defeated. Reporting them separately, with the metrics appropriate to each, avoids the error of reading an evasion-resistance result as a population estimate or vice versa.

3.7 Containment and the Handling of Real Malware

Because part of the corpus consists of real malicious samples, containment was treated as a first-class methodological concern rather than an afterthought. Two dangerous real polyglots — a JPEG-plus-ZIP and a PNG-plus-RAR sample associated with known malware families — are deliberately not downloaded at all; an inert equivalent that reproduces the structural evasion without the live payload is built instead, and this containment decision is documented so that the choice is transparent. Samples that are downloaded are handled only under supervisor approval and only within the hardened host configuration described next.

3.8 Accepted File Types and Threat Scope

SecureUploader accepts a deliberately narrow set of file types. Office documents and archive formats are excluded because their macro and container semantics broaden the attack surface beyond what the pipeline is designed to neutralize. The accepted types and their in-scope threats are listed in Table 9.

Type	Extensions	In-scope threats
Raster image	.jpg .jpeg .png .webp	Polyglots, appended payloads, parasite chunks
Vector image	.svg	Script, event handlers, foreignObject, XXE
Document	.pdf	Nested-filter payloads, embedded links / actions

Table 9: Accepted file types and their in-scope threat classes.

GIF was removed from an earlier accepted set and WebP added; no Office or archive types are accepted. Because the accepted types are almost entirely non-executable on the host operating system, this narrow scope is also what makes controlled testing on the host machine defensible: a sample that cannot execute on the host cannot compromise it merely by being present.

3.9 Testing Environment and Host Controls

A full virtual machine proved unusably slow on the available hardware, so testing is performed on the host with layered controls that together substitute for isolation. Samples are neutralized on disk by appending an inert suffix that the harness strips at scan time, so a stored sample is never a live file of its claimed type; folder previews, thumbnails, and search indexing are disabled for the lab directory so that no background process opens a sample; automatic sample submission is turned off; and real-time protection remains enabled everywhere except for a single narrowly-scoped exclusion on the scanner's temporary directory.

That one exclusion resolved a subtle measurement error worth stating in full, because it is also reported as a self-bypass finding in Chapter Six. The recursive PDF-inflation layer writes decompressed streams to a temporary file before scanning them; the host antivirus quarantined EICAR payloads in that temporary file before YARA could read them, causing genuine detections to be recorded as failures — the pipeline was working, but the environment was hiding its successes. Routing all scanner temporary files to a single excluded directory, with real-time protection otherwise on everywhere, removed the interference without weakening host protection.

3.9.1 Determinism and Reproducibility

A property that runs through both the design and the evaluation is determinism: the pipeline uses no randomness, no time-dependent behavior, and no learned model, so the same input always produces the same verdict and, for a neutralized file, the same rewritten bytes. This is what makes the evaluation reproducible — a reported verdict can be regenerated exactly — and it is what allows the pipeline to be used inside an automated build without introducing nondeterministic failures.

Determinism also disciplines the neutralization layers. Because re-encoding an image or sanitizing an SVG must yield identical output for identical input, the transformation cannot depend on any ambient state, which in turn makes the transformed file safe to cache and safe to compare byte-for-byte across runs. The 28,554-to-14,863-byte reduction reported for the appended-payload

fixture in Chapter Five is not an average over runs but a fixed result that the same input reproduces every time.

3.10 Evaluation Metrics

Results in Chapter Five are reported with the standard classification metrics, defined in Table 10 in terms of true and false positives and negatives (TP, FP, TN, FN). A positive is a file the pipeline flags as malicious; recall measures how many malicious files are caught, precision how many flagged files are truly malicious, and the false-positive rate how many clean files are wrongly flagged. Reporting all of them together, rather than a single headline number, is what makes the trade-off between coverage and false positives visible.

Metric	Definition	Formula
Accuracy	Fraction of all files classified correctly	(TP+TN) / (TP+TN+FP+FN)
Precision	Fraction of flagged files that are truly malicious	TP / (TP+FP)
Recall	Fraction of malicious files that are caught	TP / (TP+FN)
Specificity	Fraction of clean files correctly accepted	TN / (TN+FP)
F1-score	Harmonic mean of precision and recall	2·P·R / (P+R)
False-positive rate	Fraction of clean files wrongly flagged	FP / (FP+TN)

Table 10: Definitions of the evaluation metrics used in Chapter Five.

3.11 Chapter Summary

SecureUploader is built on a memory-safe, middleware-oriented platform with a small set of mature libraries, under an explicit threat model in which the attacker controls the uploaded bytes and acts only through the endpoint. It is evaluated on a balanced public corpus and a hand-built adversarial corpus, restricted to a narrow set of near-non-executable file types, tested under a hardened host configuration whose one measurement pitfall was identified and corrected, and reported with the standard classification metrics. Chapter Four turns to the design of the pipeline itself.

Chapter Four — System Design and Implementation

4.1 Introduction

This chapter presents the design and implementation of the SecureUploader pipeline: its guiding principles, the abstraction and orchestration that let independent layers compose, the nine layers themselves grouped by function and examined individually, the caching layer that makes repeated scanning cheap, the security-headers middleware and configuration that govern its behavior, the verdict and evidence-trail model, and the frontend through which its decisions are inspected.

4.2 Design Principles

Five principles govern the design.

- **Defense in depth.** No file is trusted until it has passed every applicable layer, and each layer reasons about a different property, so that defeating the pipeline requires defeating all applicable layers at once.
- **Fail closed.** Ambiguity resolves toward safety: a file that cannot be shown safe is rejected or neutralized rather than accepted, and an error in a layer stops the pipeline rather than letting the file through.
- **Ordering by cost.** Layers run from cheapest and most general to most expensive and most specific, so most rejected files are eliminated early and the costly layers run only on inputs that survive the earlier gates.
- **Determinism.** The same input always yields the same verdict and, for neutralization, the same rewritten output, which is a prerequisite for reproducibility and for use inside automated pipelines.
- **Transparency.** Every verdict carries a per-layer evidence trail, so a decision is auditable rather than an opaque score.

4.3 High-Level Architecture

The pipeline comprises nine layers in three functional groups: structural layers that check metadata and format coherence, detection layers that scan decoded content for malicious patterns, and transformation layers that rewrite a file to neutralize a payload. The overall data flow, from an untrusted upload to one of three verdicts, is shown in Figure 5, and the layers are enumerated in Table 11.

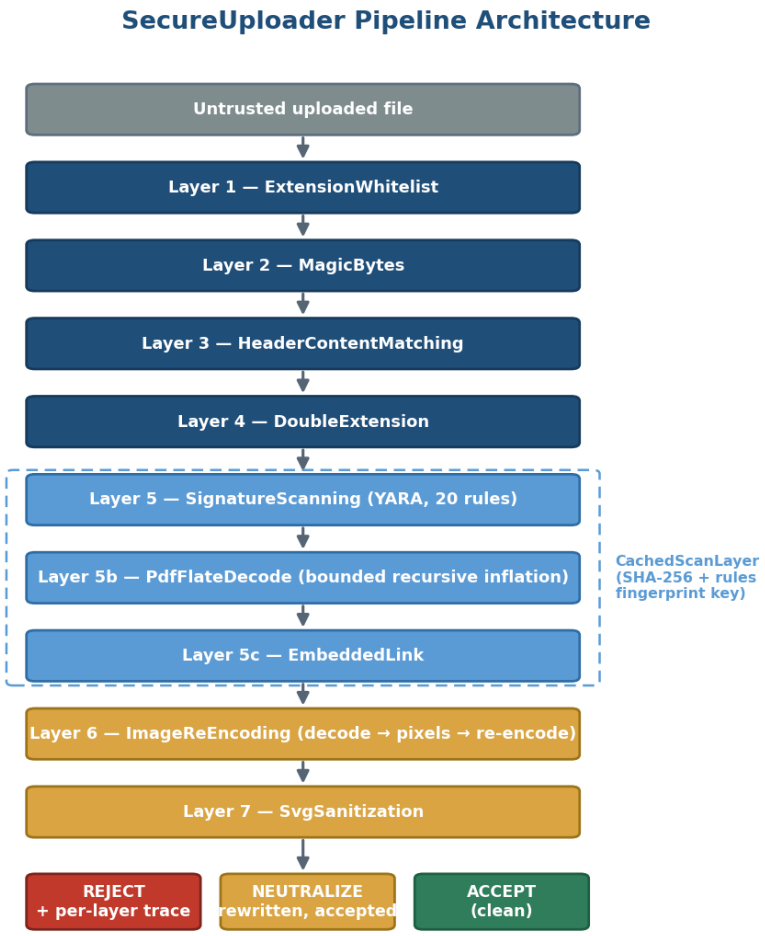


Figure 5: SecureUploader pipeline architecture

#	Layer	Group	Responsibility
1	ExtensionWhitelist	Structural	Reject disallowed extensions
2	MagicBytes	Structural	Match leading bytes to the claimed type
3	HeaderContentMatching	Structural	Check header / content coherence
4	DoubleExtension	Structural	Detect double / disguised extensions
5	SignatureScanning	Detection	YARA scan (20 rules, incl. EICAR)
5b	PdfFlateDecode	Detection	Bounded recursive stream inflation, then scan
5c	EmbeddedLink	Detection	Inspect embedded links / actions
6	ImageReEncoding	Transformation	Decode to pixels and re-encode (neutralize)
7	SvgSanitization	Transformation	Strip script, handlers, foreignObject, XXE

Table 11: The nine inspection layers, their category, and their responsibility.

4.4 The IFileScanner Abstraction and Orchestration

Every layer implements a single interface, IFileScanner, exposing one operation that takes the file bytes and the accumulating trace and returns a per-layer result. This uniformity is what lets the layers compose: the orchestrator holds an ordered list of scanners and runs each in turn, as shown in Figure 6. A layer may pass the file to the next, reject it and short-circuit the remainder of the pipeline, or — for the transformation layers — rewrite the bytes and pass the neutralized result onward. Because rejection short-circuits, the expensive detection and transformation layers never run on a file already eliminated by a cheap structural check.

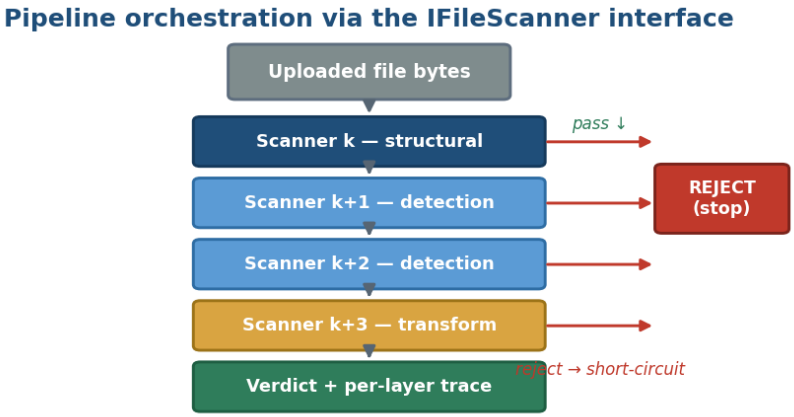


Figure 6: Pipeline orchestration via IFileScanner

The same abstraction underpins evaluation. The reference validators used in Chapter Five — the DVWA-style scanners and the basic baseline — also implement IFileScanner, so the comparison orchestrator can run the identical bytes through every validator in turn and record each one's verdict and trace under identical conditions.

```
public interface IFileScanner
{
    string     LayerName { get; }
    ScanResult Scan(FileContext file, ScanTrace trace);
}

public sealed record ScanResult(
    Verdict Verdict,          // Pass, Reject, Neutralize
    string  Reason,           // human-readable evidence
    byte[]? RewrittenBytes);  // set when Verdict == Neutralize
```

Listing 1: The IFileScanner interface and the per-layer result type.

```
public PipelineResult Run(FileContext file)
{
    var trace = new ScanTrace();
    foreach (var scanner in _scanners)      // ordered: cheap -> expensive
    {
        var sw = Stopwatch.StartNew();
        var r  = scanner.Scan(file, trace);
        trace.Record(scanner.LayerName, r, sw.Elapsed);

        if (r.Verdict == Verdict.Reject)
            return PipelineResult.Rejected(trace);   // short-circuit
        if (r.Verdict == Verdict.Neutralize)
            file = file.WithBytes(r.RewrittenBytes!); // pass rewritten on
    }
    return PipelineResult.Accepted(file, trace);
}
```

Listing 2: The orchestration loop: ordered execution with short-circuit on rejection.

The interface, shown in Listing 1, keeps every layer to the same shape: it names itself, receives the file and the accumulating trace, and returns a verdict together with any rewritten bytes. The record type distinguishes the three possible outcomes without any layer needing to know about the others.

```
public interface IFileScanner
{
    string     LayerName { get; }
    ScanResult Scan(FileContext file, ScanTrace trace);
}

public sealed record ScanResult(
    Verdict Verdict,          // Pass, Reject, Neutralize
    string  Reason,           // human-readable evidence
    byte[]? RewrittenBytes);  // set when Verdict == Neutralize
```

Listing 3: The IFileScanner interface and the ScanResult verdict record.

The orchestrator, shown in Listing 2, is deliberately small: it iterates the ordered scanners, times each one, records it in the trace, short-circuits on the first rejection, and threads any rewritten bytes into the next layer so that a neutralization is visible to everything downstream.

```
public PipelineResult Run(FileContext file)
{
    var trace = new ScanTrace();
    foreach (var scanner in _scanners)      // ordered: cheap -> expensive
    {
        var sw = Stopwatch.StartNew();
        var r  = scanner.Scan(file, trace);
        trace.Record(scanner.LayerName, r, sw.Elapsed);

        if (r.Verdict == Verdict.Reject)
            return PipelineResult.Rejected(trace);    // short-circuit
        if (r.Verdict == Verdict.Neutralize)
            file = file.WithBytes(r.RewrittenBytes!);  // pass rewritten on
    }
    return PipelineResult.Accepted(file, trace);
}
```

Listing 4: The orchestration loop: ordered execution, short-circuit, and trace.

4.4.1 Extending the Pipeline with a New Layer

The uniform interface makes the pipeline straightforward to extend. Adding a new inspection is a matter of implementing IFileScanner and registering the implementation at the appropriate position in the ordered list; no existing layer needs to change, because each layer is oblivious to the others and communicates only through the shared result type and trace. This is a direct benefit of the abstraction: a new defense for a new format, or a stricter check for an existing one, can be developed and tested in isolation and then slotted into the sequence at a point consistent with its cost.

The same independence is what makes the ablation study planned in Chapter Seven possible. Because a layer can be removed from the ordered list without disturbing the rest, the marginal contribution of any single layer to detection and to neutralization can be measured by running the corpus with and without it. The architecture thus serves not only the running system but also the empirical study of the system, which is a deliberate design goal rather than an accident.

4.5 Structural Layers (1–4)

The first four layers reason about metadata and format structure without decoding content, and are cheap enough to run on every upload.

4.5.1 ExtensionWhitelist (Layer 1)

The first layer rejects any file whose extension is outside the accepted set. It is trivially bypassable in isolation, but as the first gate it removes the large class of obviously-disallowed types and fixes which parsers the later layers must consider, so that subsequent layers can make format-specific assumptions safely.

4.5.2 MagicBytes (Layer 2)

The second layer reads the leading bytes and verifies that they match the claimed type — a JPEG must begin with the JPEG magic number, a PNG with the PNG signature, and so on. This defeats naive extension spoofing, because a renamed script or a header-less payload does not begin with a valid image header, and it is the layer that rejects the header-less fixture in Chapter Five.

```
static readonly Dictionary<string, byte[][]> Magic = new()
{
    [".jpg"]  = new[] { new byte[]{0xFF,0xD8,0xFF} },
    [".png"]  = new[] { new byte[]{0x89,0x50,0x4E,0x47} },
    [".webp"] = new[] { "RIFF"u8.ToArray() },   // plus "WEBP" at offset 8
    [".pdf"]  = new[] { "%PDF-"u8.ToArray() },
};

bool MagicMatches(string ext, ReadOnlySpan<byte> head) =>
    Magic.TryGetValue(ext, out var sigs) &&
    sigs.Any(sig => head[..sig.Length].SequenceEqual(sig));
```

Listing 5: Magic-byte validation against the claimed extension (Layer 2).

4.5.3 HeaderContentMatching (Layer 3)

The third layer checks that the structure implied by the header is consistent with the rest of the file: that declared lengths and offsets are coherent and that trailing markers appear where the format requires. The structural inconsistencies introduced by stacking two formats or appending an archive are surfaced here, before any expensive decoding is attempted.

```
bool HeaderContentCoherent(FileContext f)
{
    if (ReadDeclaredLength(f.Head) is long len && len != f.Bytes.Length)
        return false;                    // data appended after declared end
    if (!TrailingMarkerPresent(f))       // e.g. JPEG EOI, PNG IEND
        return false;
    return true;
}
```

Listing 6: Header–content coherence check that surfaces appended data (Layer 3).

4.5.4 DoubleExtension (Layer 4)

The fourth layer detects disguised names such as a script extension hidden before an allowed one, closing the double-extension bypass against servers that key execution off an interior extension. Together the four structural layers eliminate the cheapest and most common evasions and narrow the input reaching the detection layers to well-formed files of an accepted type.

4.5.5 Error Handling and Fail-Closed Behavior

Parsing untrusted input is inherently error-prone, and how the pipeline responds to an error is a security decision in its own right. The pipeline fails closed: if a layer throws while inspecting a file — a malformed structure that crashes a parser, an inflation that trips a resource limit, an image that cannot be decoded — the file is rejected rather than accepted, and the error is recorded in the trace with the layer that produced it. An attacker therefore cannot benefit by crafting an input that breaks a layer, because a broken layer yields a rejection, not a pass.

This posture extends to resource limits. The depth and expansion bounds on inflation, the pixel bound on re-encoding, and the size limits on upload are all enforced as hard limits whose violation

is a rejection, so that no single file can consume unbounded work. The combination of fail-closed error handling and hard resource limits is what allows the pipeline to inspect hostile input safely: a file can be malformed, oversized, or actively adversarial, and the worst outcome is a recorded rejection rather than a crash or an unbounded computation.

4.6 Detection Layers (5, 5b, 5c)

4.6.1 YARA Signature Scanning (Layer 5)

Layer 5 scans the file with a curated YARA rule set of twenty rules, including a rule for the EICAR test string. Signatures match known-malicious markers — embedded scripts, shell tokens, document actions, and known payload fragments — anywhere in the content it is given. The rule categories are summarized in Table 12. Because signature scanning is only as good as the content it is shown, its effectiveness against concealed payloads depends on the inflation performed by the next layer.

Table 12: Categories of YARA rules in the signature-scanning layer.

Category	Example targets
Test markers	EICAR test string
Web shells	PHP / ASP / JSP shell tokens
Script injection	script elements, event-handler attributes
PDF actions	JavaScript, OpenAction, Launch actions
Archive markers	embedded ZIP / RAR signatures
Obfuscation	suspicious encodings and patterns

4.6.2 PDF FlateDecode Recursive Inflation (Layer 5b)

Layer 5b addresses nested-filter concealment. A PDF stream may be encoded by a chain of FlateDecode filters; a scanner that decodes only the outermost layer never sees the payload. This layer performs bounded, recursive inflation to a configurable depth, expanding nested filters before handing the decoded content to signature scanning, while enforcing limits on depth and on expanded size that prevent a decompression bomb from exhausting memory.

Two fixtures demonstrate the effect and are analyzed in Chapter Five: an EICAR payload behind a single FlateDecode is caught after one level of inflation, and the same payload behind a double FlateDecode is caught after two — the latter having passed plain signature scanning, which is

precisely the ablation argument for why this layer exists as a separate stage rather than being folded into Layer 5.

```
byte[] InflateRecursive(byte[] data, int depth, InflateLimits lim)
{
    if (depth > lim.MaxDepth) throw new DepthExceeded();
    if (!LooksFlateEncoded(data)) return data;

    var inflated = Inflate(data);                 // one FlateDecode pass
    if (inflated.Length > lim.MaxExpandedBytes)   // decompression-bomb guard
        throw new ExpansionTooLarge();

    // peel nested filters one level at a time, up to MaxDepth
    return InflateRecursive(inflated, depth + 1, lim);
}
```

Listing 7: Bounded recursive inflation with depth and expansion guards (Layer 5b).

The inflation routine, sketched in Listing 3, is where the two safety bounds live. It refuses to recurse past a configured maximum depth and refuses to accept an inflated result larger than a configured maximum size, so a maliciously nested or explosively compressed stream is stopped rather than allowed to exhaust memory.

```
byte[] InflateRecursive(byte[] data, int depth, InflateLimits lim)
{
    if (depth > lim.MaxDepth) throw new DepthExceeded();
    if (!LooksFlateEncoded(data)) return data;      // nothing left to peel

    var inflated = Inflate(data);                    // one FlateDecode pass
    if (inflated.Length > lim.MaxExpandedBytes)      // decompression-bomb guard
        throw new ExpansionTooLarge();

    return InflateRecursive(inflated, depth + 1, lim); // peel the next layer
}
```

Listing 8: Bounded recursive inflation with depth and expansion guards.

4.6.3 Embedded-Link Inspection (Layer 5c)

Layer 5c inspects embedded links and actions — the mechanisms by which a document can reference or trigger external resources — and flags those that indicate malicious intent, such as actions that fetch or launch external content. It closes the gap between a document that contains no malicious bytes of its own and one that reaches out to fetch them.

```
IEnumerable<Finding> InspectLinks(PdfDocument pdf)
{
    foreach (var action in pdf.Actions)
        if (action.Type is "/Launch" or "/URI" or "/GoToR")
            yield return new Finding(action.Type, action.Target);
}
```

Listing 9: Embedded-link and action inspection for documents (Layer 5c).

4.7 Transformation Layers (6, 7)

4.7.1 Image Re-encoding (Layer 6)

Layer 6 replaces an earlier metadata-inspection layer with an active defense. It decodes an accepted raster image to its raw pixels and re-encodes it from scratch using Magick.NET, discarding every byte that is not part of the pixel data. Appended archives, parasite metadata chunks, and cavity payloads are all destroyed while the visible image is preserved, and decompression-bomb guards bound the work performed so that a maliciously large image cannot turn the defense into a denial-of-service vector.

This is the primary neutralization mechanism for image polyglots: because, as Chapter Two established, the malicious second format is never part of the pixel data, rebuilding from pixels removes it regardless of the polyglot construction used. The file is accepted, but only after being rewritten into a provably inert form — and the rewrite is deterministic, so the same input always yields the same neutralized output.

```
byte[] ReEncode(byte[] input, ImageFormat fmt, ReEncodeLimits lim)
{
    using var img = new MagickImage(input);       // decode to pixels
    Guard(img.Width * img.Height <= lim.MaxPixels); // bomb guard
    img.Strip();                                  // drop metadata chunks
    img.Format = fmt;
    return img.ToByteArray();                     // re-encode from pixels only
}
```

Listing 10: Image re-encoding: rebuild the file from decoded pixels only (Layer 6).

Rebuilding from pixels removes everything outside the pixel data, but it does not by itself disturb the pixels, so a payload hidden in the least-significant bits of the pixel values themselves survives a lossless re-encode. For deployments that must also close this pixel-domain steganography channel, the layer exposes an optional clearing step, controlled by the ClearLeastSignificantBit configuration flag, that zeroes the least-significant bit of every colour channel by a bitwise-AND with 0xFE before re-encoding. Because it alters each pixel by at most one intensity level, the step is visually imperceptible; its effect on hidden data, and the reason it is offered as an option rather than applied unconditionally, are evaluated in Section 5.6.

The re-encode, shown in Listing 4, is short by design: decode to a pixel buffer, guard the pixel count against a decompression bomb, strip any residual metadata, set the target format, and serialize from the pixels alone. Nothing that was not a pixel survives the call.

```
byte[] ReEncode(byte[] input, MagickFormat fmt, ReEncodeLimits lim)
{
    using var img = new MagickImage(input);   // decode to pixels
    Guard(img.Width * img.Height <= lim.MaxPixels);
    img.Strip();                              // drop metadata chunks
    img.Format = fmt;
    return img.ToByteArray();                 // re-encode from pixels only
}
```

Listing 11: Image re-encoding: rebuild the file from decoded pixels.

4.7.2 SVG Sanitization (Layer 7)

Layer 7 parses an SVG and removes its active content while preserving the vector drawing. The constructs it strips, and the risk each poses, are listed in Table 13. As with image re-encoding, the outcome is a neutralized, accepted file rather than an outright rejection, so a legitimate vector graphic that merely happened to carry a handler is preserved rather than discarded.

Table 13: SVG constructs removed by the sanitization layer and the risk each poses.

Construct	Risk	Action
script element	Stored XSS	Removed
Event handlers (onload, onclick, …)	Stored XSS	Removed
foreignObject	Arbitrary embedded HTML / JS	Removed
External-entity declarations	XXE, SSRF, file disclosure	Removed
External references (href to remote)	Data exfiltration, SSRF	Neutralized
Vector shapes and paths	None	Preserved

```
XDocument Sanitize(XDocument svg)
{
    svg.Descendants("script").Remove();
    svg.Descendants("foreignObject").Remove();
    foreach (var el in svg.Descendants())
        el.Attributes()
          .Where(a => a.Name.LocalName.StartsWith("on"))  // onload, onclick
          .Remove();
    StripExternalEntities(svg);                           // XXE / SSRF
    return svg;
}
```

Listing 12: SVG sanitization: strip active content, preserve the vector drawing (Layer 7).

The sanitizer, sketched in Listing 5, removes the dangerous nodes and attributes and neutralizes external references while leaving the drawing untouched, so the returned document renders identically but can no longer execute.

```
XDocument Sanitize(XDocument svg)
{
    svg.Descendants("script").Remove();
    svg.Descendants("foreignObject").Remove();
    foreach (var el in svg.Descendants())
        el.Attributes()
          .Where(a => a.Name.LocalName.StartsWith("on"))  // onload, onclick
          .Remove();
    StripExternalEntities(svg);                            // XXE / SSRF
    return svg;                                            // drawing preserved
}
```

Listing 13: SVG sanitization: strip active content, preserve the vector drawing.

4.8 Caching Subsystem

The detection layers are the most expensive in the pipeline, and identical files are frequently re-uploaded. A CachedScanLayer wraps layers 5, 5b, and 5c behind a content-addressed cache whose key has three parts, shown in Figure 7: the layer name, the SHA-256 of the file content, and a fingerprint of the YARA rules file. Including the rules fingerprint means the cache invalidates immediately whenever the rules change, so a stale rule set can never mask a new detection — a failure mode discussed in Chapter Six. A repeated upload of an identical file returns the cached verdict and drops the scan cost to near zero, and the interface surfaces a FromCache flag so a cached verdict is never mistaken for a fresh scan.

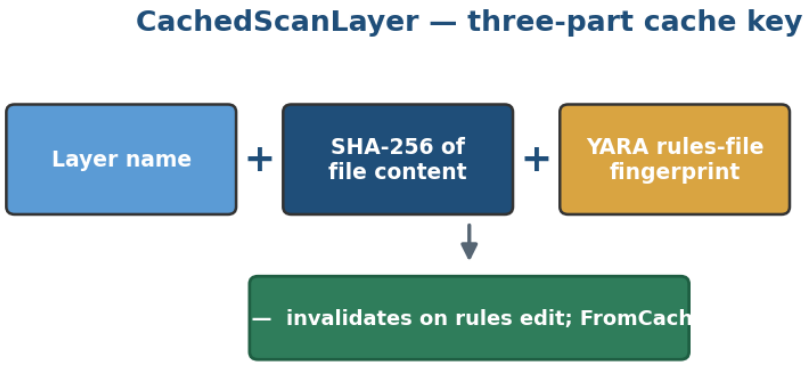


Figure 7: CachedScanLayer three-part cache key

It is worth stating explicitly that the SHA-256 in the system is used only as a cache key. There is no hash database of known-malicious files; detection comes from the layers themselves, not from matching a stored list of hashes. Any description of the system that implies a hash-based signature database would be incorrect.

```
string CacheKey(string layer, byte[] content, string rulesPath)
{
    var contentHash = SHA256(content);
    var rulesFinger = SHA256(File.ReadAllBytes(rulesPath));
    // any change to the rules file yields a new key -> automatic invalidation
    return $"{layer}:{contentHash}:{rulesFinger}";
}
```

Listing 14: The three-part cache key that ties a cached verdict to the current rule set.

Listing 6 shows how the three-part key is formed. Because the rules-file fingerprint is part of the key, any edit to the rules produces a different key for every file, which is exactly what makes a stale cached verdict impossible after a rule change.

```
string CacheKey(string layer, byte[] content, string rulesPath)
{
    var contentHash = Sha256(content);
    var rulesPrint  = Sha256(File.ReadAllBytes(rulesPath));
    return $"{layer}:{contentHash}:{rulesPrint}"; // rules change -> new key
}
```

Listing 15: The three-part, rules-aware cache key.

4.9 Configuration Model

The pipeline's behavior is governed by a set of runtime configuration sections, allowing thresholds, depths, and modes to be tuned without code changes. The principal sections are listed in Table 14.

Section	Governs
Upload	Accepted extensions and size limits
Yara	Rule-set path and scanning options
PdfDecode	Recursive inflation depth and bomb guards
ReEncoding	Image re-encoding parameters
Cache	Cache behavior and invalidation
Demo	Comparison endpoints on / off (server-side switch)

Table 14: Runtime configuration sections exposed by the pipeline.

A server-side demo switch gates the comparison endpoints used for evaluation; when it is off, those endpoints return a not-found rather than a forbidden response, so their existence is not disclosed in production. This switch must be set off before deployment, and treating it as configuration rather than a compile-time constant keeps that decision explicit.

4.10 Security-Headers Middleware

Independently of the inspection layers, a security-headers middleware sets the response headers that harden how a served file is interpreted by a browser — constraining content types and framing so that even an accepted file is served under a restrictive policy. This is a defense-in-depth

complement to neutralization: the pipeline first removes active content from a file, and the headers then reduce the impact of anything that might slip through, so the two mechanisms reinforce rather than duplicate each other.

4.11 Verdicts and the Evidence Trail

Every file resolves to one of three verdicts: rejected, neutralized (accepted after rewriting), or accepted clean. Regardless of verdict, the pipeline records a per-layer trace with wall-clock timings, so that for any file it is possible to see exactly which layers ran, which layer acted, and why. This reproducible evidence trail is the transparency contribution of the system: a verdict is never an opaque score but an auditable sequence of decisions, each attributable to a named layer and a specific piece of evidence.

4.11.1 Reading a Trace

A trace is read top to bottom as the file's journey through the pipeline. Each entry names a layer, the result it returned — pass, reject, or neutralize — and, where relevant, the evidence and the time the layer took. For an accepted-clean file every entry is a pass; for a rejected file the trace passes through the early layers and terminates at the layer that rejected, with the reason recorded there and no later layers run, because rejection short-circuits the remainder; for a neutralized file the trace shows a transformation layer rewriting the bytes, with the before-and-after sizes recorded, and the file continuing to acceptance. Appendix B shows the concrete format of both a rejected and a neutralized trace.

Two fields in the trace are especially useful in practice. The acting-layer field names the single layer responsible for the verdict, so an analyst can see at a glance why a file was blocked or rewritten without reading the whole trace. The from-cache flag records whether the detection result was served from the content cache rather than freshly computed, so a fast verdict is never mistaken for a skipped one. Together these turn the trace from a log into an explanation that a non-specialist can follow.

4.12 The Instrument-Panel Frontend

The decisions of the pipeline are inspected through a React frontend styled as a dark instrument panel. It renders the per-layer trace as a pipeline rail that stops at the acting layer, colors the verdict in one of three ways — accepted, rejected, or neutralized — and, in evaluation mode, presents a

comparison panel that shows the verdicts of all six reference validators on the same file side by side. The frontend is not merely cosmetic: by making the evidence trail visible it turns the transparency principle into something an analyst can actually use, and it is the surface on which the FromCache flag and the per-layer timings are read.

4.14 Worked Example: A File's Journey Through the Pipeline

To make the interaction of the layers concrete, consider a single crafted input: a file named photo.png that carries a genuine PNG header, valid pixel data, and — hidden in an ancillary text chunk — a small PHP web shell. Tracing this file through the pipeline shows how the layers combine to produce a safe outcome that no single layer could reach alone.

At Layer 1 the extension .png is on the allow-list, so the file passes. At Layer 2 the leading bytes are the genuine PNG signature, so magic-byte validation also passes — this is the point at which a magic-byte-only validator would accept the file and stop. At Layer 3 the header and the chunk structure are internally coherent, so header–content matching finds nothing wrong, and at Layer 4 the single, legitimate extension raises no double-extension flag. The four structural layers, in other words, all pass, exactly as the attacker intended when they built a structurally valid PNG.

At Layer 5 the YARA scan inspects the file's bytes and may match the PHP web-shell pattern in the text chunk; if the payload is obfuscated enough to slip the signatures, the file still carries it forward. Because the file is a PNG rather than a PDF, the PDF-specific layers 5b and 5c pass it through. The decisive moment is Layer 6: the image is decoded to its pixels and re-encoded from scratch, and the text chunk — which is not pixel data — is simply not part of the rebuilt file. The web shell is gone. The verdict is neutralized-and-accepted, the file remains a valid, viewable PNG, and the per-layer trace records that image re-encoding was the acting layer.

The lesson of the walkthrough is that the file defeated every structural check and possibly the signature scan, yet was still rendered harmless, because the transformation layer removed the payload by construction rather than by recognizing it. This is defense in depth working as intended: the attacker satisfied the properties the early layers check, but could not satisfy the one the transformation layer enforces — that the accepted file consist of nothing but its pixels.

4.15 Chapter Summary

SecureUploader composes nine independent layers — four structural, three detection, and two transformation — behind a single IFileScanner abstraction, in order of increasing cost and specificity, wraps the detection layers in a rules-aware content cache, hardens served responses with security headers, exposes its behavior through configuration, and records an auditable per-layer evidence trail rendered by an instrument-panel frontend. Chapter Five evaluates this design empirically.

Chapter Five — Experimental Results

5.1 Introduction

This chapter evaluates SecureUploader along two axes. The first is aggregate detection performance on a large, realistic public corpus, reported as a confusion matrix and standard metrics with an analysis of both the false positives and the false negatives. The second is evasion resistance on a hand-built adversarial corpus, where the pipeline is compared, fixture by fixture, against five reference validators, followed by a per-sample analysis of the most instructive cases, a note on timing, and a contextual comparison with machine-learning detectors and related systems.

5.2 Evaluation Setup

Aggregate performance is measured on the full CIC-Evasive-PDFMal2022 corpus of 10,012 files — 5,549 malicious and 4,463 clean PDF documents. Evasion resistance is measured on the adversarial corpus described in Chapter Three, comprising real polyglots, crafted inert fixtures, and evasive documents whose expected behavior is fully known. Metrics follow the definitions in Chapter Three, and every verdict is produced by the same pipeline configuration so that results are directly comparable.

5.3 Aggregate Results

5.3.1 Confusion Matrix and Metrics

On the full 10,012-file corpus the pipeline correctly classified 5,298 of 5,549 malicious files and 4,067 of 4,463 clean files, giving 95.48% recall, 93.05% precision, 93.54% accuracy, and a 94.25% F1 score, with a specificity of 91.13%. The confusion matrix and the derived metrics are shown in Figure 8, and the four headline metrics are shown together in Figure 9.

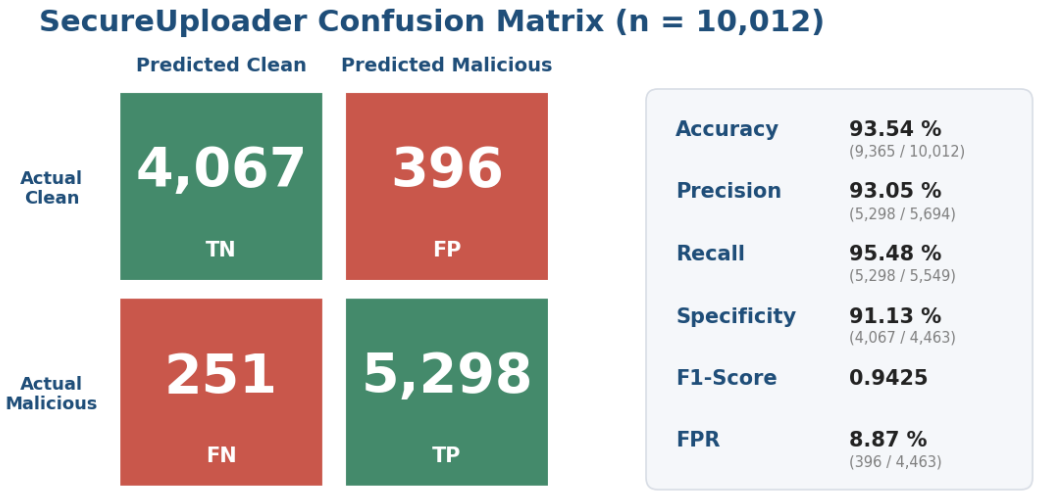


Figure 8: Confusion matrix and performance metrics

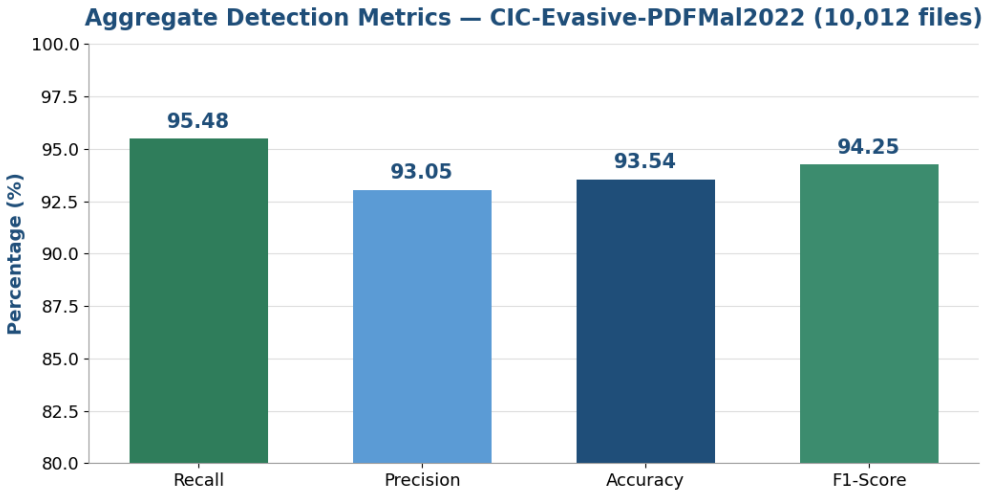


Figure 9: Aggregate detection metrics

These results place the pipeline in a strong position for a training-free, byte-level system. Recall above 95% means fewer than five malicious files in a hundred pass undetected, and a sub-9% false-positive rate on a corpus whose clean half was engineered to resemble the malicious class is low enough to be workable. That the four metrics are close together indicates the pipeline is not trading one against another — it is neither over-aggressive nor over-permissive — which is the balance a deployable validator needs.

5.3.2 Interpretation

It is worth reading the numbers against the operating regime. The pipeline achieves this without any training phase and without extracting a feature vector: it acts on raw bytes and reaches a verdict through a fixed sequence of layers. A recall of 95.48% on a corpus explicitly built to include evasive PDFs indicates that the detection layers, and in particular the recursive inflation that exposes concealed payloads, are doing substantive work rather than catching only the easy cases.

5.3.3 False-Positive Analysis

The false-positive result is best understood against the way the clean set was built. Because the corpus selects benign documents to resemble the malicious class, medium-severity signature and structural findings fire on a large fraction of clean files: an initial full-corpus run, before any severity threshold was applied, rejected 4,285 of the 4,463 clean files, a false-positive rate of 96.0%. That figure is a property of the dataset rather than of the pipeline — the clean half is adversarial by construction — but it is also directly actionable. Introducing a severity-threshold gate, which rejects only high- and critical-severity findings while logging lower-severity ones, reduces the false positives to 396 of 4,463, an 8.87% false-positive rate on the same clean set, with no material change to true detections (recall moves only from 95.8% to 95.48%). Figure 10 shows the two rates side by side. The residual 396 false positives are dominated by encrypted, unscannable PDFs that the pipeline fails closed on, together with a small tail of documents whose structure genuinely matches a high-severity rule; the former are the target of a further rule downgrade, the latter are the irreducible cost of a fail-closed posture on a corpus designed to blur the two classes.

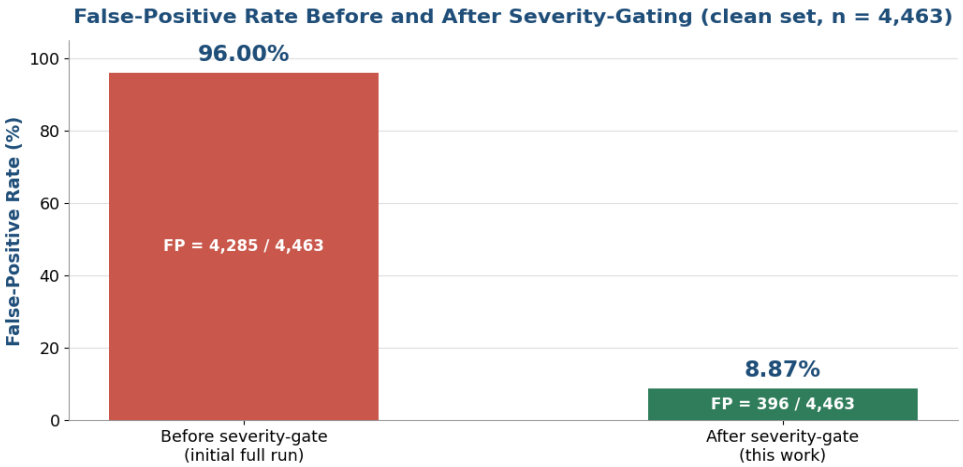


Figure 10: False-positive rate before and after severity-gating

5.3.4 False-Negative Analysis

The 251 malicious files that passed undetected — 4.5% of the malicious set — are the complement worth examining. They correspond to malicious PDFs whose indicators fall outside the current rule set and are not exposed by inflation — payloads that neither match a signature nor sit behind a FlateDecode chain the layer reaches. This is the expected failure mode of a signature-and-structure approach with no learned component: it catches what its rules and its structural checks describe, and misses novel indicators for which no rule yet exists. Growing the rule set and the adversarial corpus, and the planned ablation study, are the mechanisms by which this residue is intended to be reduced.

5.4 Adversarial-Corpus Comparison

Aggregate accuracy on realistic documents does not by itself demonstrate evasion resistance, because a corpus of real-world files contains few inputs engineered specifically to bypass a given check. The adversarial corpus fills that gap. Each fixture is run through six validators: the four DVWA reference scanners (Low, Medium, High, Impossible — re-implemented in C# from their documented logic), a basic extension-plus-magic-byte baseline, and the full SecureUploader pipeline. The documented logic of each reference scanner is summarized in Table 15.

Scanner	Documented validation logic	Key weakness
DVWA Low	No validation	Accepts anything
DVWA Medium	Checks the client Content-Type header	Trusts attacker-supplied MIME
DVWA High	Extension check + image-size probe	Passes image polyglots
DVWA Impossible	Whitelist + size probe + image re-encode	No PDF or SVG handling
BasicValidation	Extension + magic-byte match	Passes valid-header polyglots
SecureUploader	Nine layers + neutralization	This work

Table 15: The six reference scanners and the documented validation logic of each.

Figure 11 shows, fixture by fixture, which validators rejected or neutralized each input, and Figure 12 summarizes the resulting detection rates.

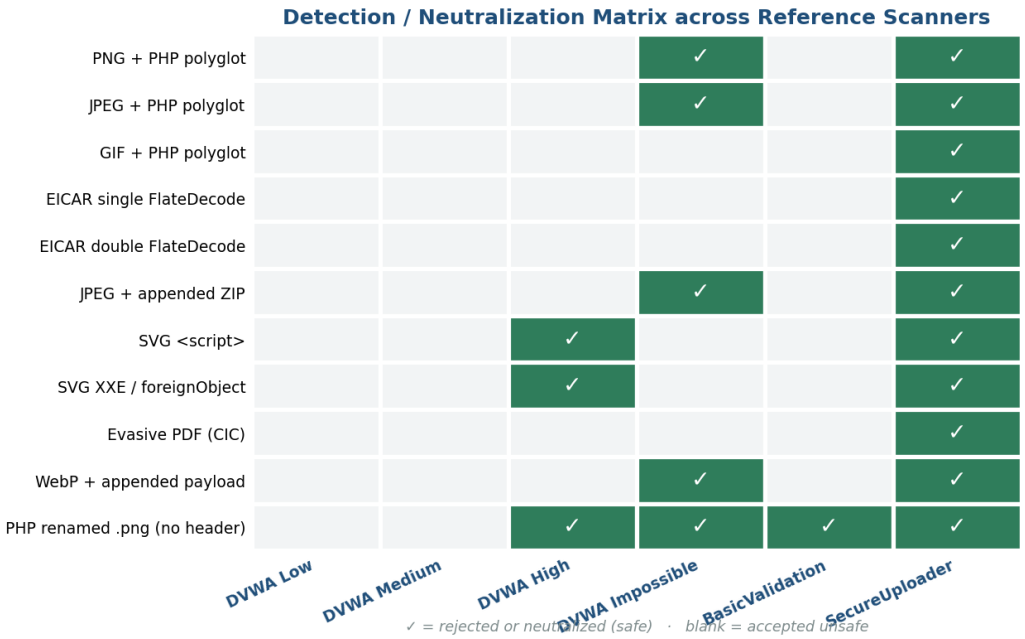


Figure 11: Detection / neutralization matrix

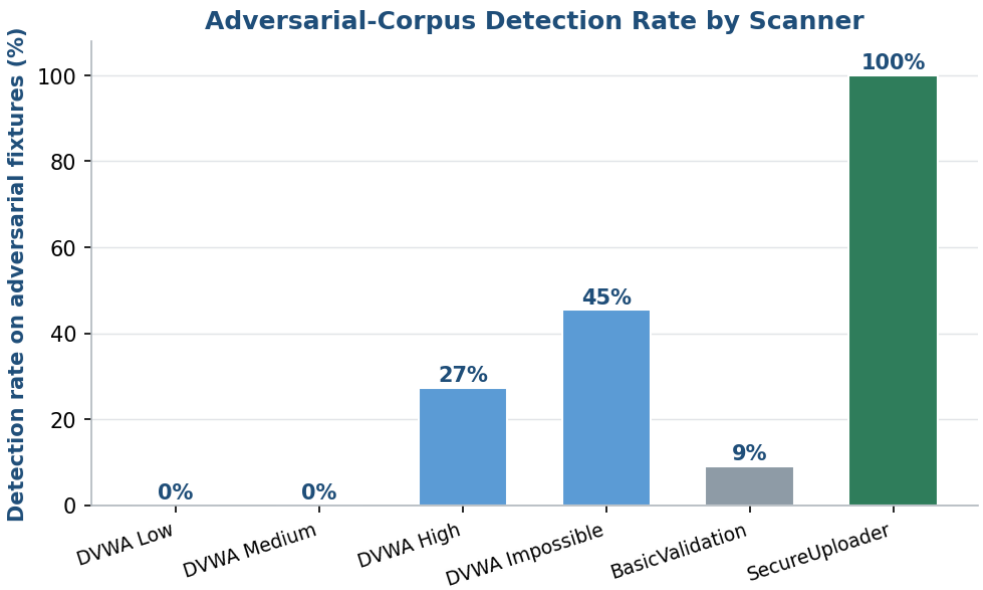


Figure 12: Adversarial-corpus detection rate by scanner

The pattern is clear. Validators that trust metadata — DVWA Low and Medium — catch none of the fixtures, because every fixture presents valid-looking metadata. Extension-plus-image checks (DVWA High) catch the SVG and header-less cases but pass every image polyglot, whose image probe succeeds. DVWA Impossible, which re-encodes images, neutralizes the image polyglots but

has no answer to nested-filter PDFs or active SVG content. Only SecureUploader rejects or neutralizes every fixture, because each evasion class is covered by a dedicated layer, exactly as the coverage map of Chapter Two predicts. The verdicts for the representative fixtures are given in Table 16.

Fixture	SecureUploader verdict	Acting layer
PNG + PHP polyglot	Neutralized	Image re-encoding (6)
JPEG + appended ZIP	Neutralized	Image re-encoding (6)
EICAR single FlateDecode	Rejected	PdfFlateDecode (5b), depth 1
EICAR double FlateDecode	Rejected	PdfFlateDecode (5b), depth 2
SVG script / onload	Neutralized	SVG sanitization (7)
SVG XXE / foreignObject	Neutralized	SVG sanitization (7)
PHP renamed .png (no header)	Rejected	MagicBytes (2)

Table 16: Per-fixture verdict of SecureUploader on representative adversarial samples.

5.5 Per-Sample Analysis

Four fixtures are examined in detail because each isolates a specific contribution of the pipeline and together they cover rejection, neutralization, and caching.

5.5.1 EICAR Behind a Single FlateDecode

A PDF carrying the EICAR test string inside a single FlateDecode stream passes plain signature scanning — the raw stored bytes do not contain the marker — but is rejected by Layer 5b after one level of inflation exposes the payload. This is the minimal demonstration that inflation must precede scanning: the very same payload is invisible before inflation and unmistakable after it, so the order of the two operations is the whole point.

5.5.2 EICAR Behind a Double FlateDecode

Wrapping the same payload in two FlateDecode passes defeats any scanner that inflates only once. Layer 5b's recursive inflation reaches the payload at the second level and rejects the file, confirming that the fix generalizes to nested rather than merely single filters. The pair of fixtures together is the ablation argument for the layer: remove recursive inflation and the double-wrapped payload passes; keep it and both are caught.

5.5.3 Appended-Payload Neutralization

A JPEG with an appended payload is not rejected but neutralized. Decoding the image to pixels and re-encoding it discards the appended bytes deterministically, reducing the file from 28,554 to 14,863 bytes as shown in Figure 13, and the same input always yields the same output. The visible image is preserved, so the neutralized file remains useful — the defining property of neutralization as distinct from rejection, and the reason a legitimate image that merely carried extra bytes is not lost to the user.

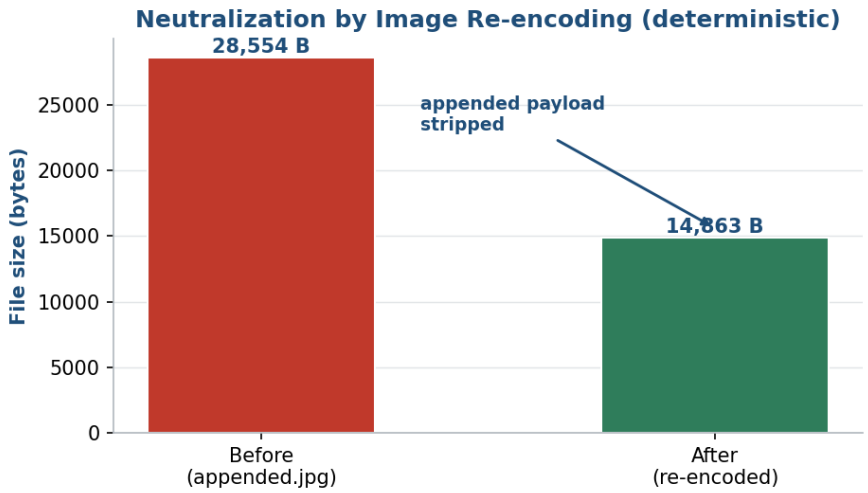


Figure 13: Neutralization by image re-encoding

5.5.4 Cache Behavior

Re-uploading an identical file returns the cached verdict and drops the scan cost to near zero, tagged in the interface as a cache hit. Because the cache key includes the YARA rules fingerprint, editing the rule set invalidates the entry immediately, so caching accelerates repeated scans without ever masking a newly-added detection — the correctness property that the stale-cache finding in Chapter Six exists to guarantee.

5.5.5 PNG Text-Chunk Parasite

A PNG carrying a payload inside an ancillary text chunk is a parasite polyglot: the payload lives in a metadata region that the image renderer ignores but that a second parser can be made to act on. Plain rendering never touches the chunk, so a validator that only confirms the image displays correctly is satisfied. The re-encoding layer, however, rebuilds the file from decoded pixels and strips ancillary chunks, so the parasite is removed and the file is neutralized — the same

mechanism that defeats appended payloads applied to an interior metadata region rather than a trailing one.

5.5.6 WebP with Appended Bytes

WebP was added to the accepted set in place of GIF, and it is subject to the same appended-payload evasion as JPEG and PNG. A WebP with bytes appended after its valid image data is neutralized by decoding and re-encoding, which discards everything outside the pixel data. Including WebP in the fixtures confirms that the re-encoding defense is not specific to a single raster format but applies uniformly across the accepted raster types, which is the property that lets a single transformation layer cover an entire class of inputs.

5.5.7 End-to-End Walkthrough of a Neutralized Upload

It is instructive to follow one file through the entire pipeline. Take the JPEG-plus-appended-ZIP polyglot of the worked example in Chapter Two, submitted to the endpoint under the name photo.jpg. The extension-whitelist layer accepts the extension; the magic-byte layer confirms the leading bytes are a valid JPEG header; the header–content-matching layer notices that the file is longer than the JPEG's declared content but, because the appended region is a coherent trailer rather than a malformed structure, allows the file to proceed to deeper inspection rather than rejecting it outright; and the double-extension layer finds no disguised interior extension.

The file then reaches the detection layers. Signature scanning finds no rule match in the raw bytes; the FlateDecode layer does not apply, because the file is not a PDF; and embedded-link inspection likewise does not apply. The file arrives at the image re-encoding layer still carrying its appended archive. There it is decoded to pixels and re-encoded from scratch, and the appended ZIP — being no part of the pixel data — is discarded. The pipeline returns a neutralized verdict, storing the rewritten image and recording a trace whose acting layer is image re-encoding and whose reason notes the size reduction.

The walkthrough illustrates the division of labor the design intends. The structural layers narrowed and characterized the file without rejecting a possibly-legitimate image; the detection layers found nothing to reject on, correctly, because the payload was inert appended data rather than a known signature; and the transformation layer resolved the ambiguity by rewriting the file into a provably inert form. No single layer handled the file alone, and the verdict is fully explained by the trace — which is the behavior the whole architecture exists to produce.

5.6 Neutralization of Pixel-Domain Steganography (LSB)

The image re-encoding layer removes payloads that live outside the pixel data — appended trailers, ancillary metadata chunks, and parasite polyglots — by rebuilding the file from its decoded pixels. One class of payload is not outside the pixels, however: least-significant-bit (LSB) steganography hides data in the low-order bit of the pixel values themselves. On a lossless format such as PNG, decoding to pixels and re-encoding preserves those values exactly, so a plain re-encode leaves an LSB payload intact. This section evaluates a dedicated countermeasure — a deterministic least-significant-bit clearing step — against a purpose-built steganography dataset, and reports it as a neutralization result rather than a detection one. The threat it addresses sits at the edge of the upload-safety model: an LSB payload is a data-exfiltration channel rather than a code-execution vector, which is why the step is offered as a defense-in-depth option rather than a core gate.

5.6.1 Dataset and Metric

The evaluation uses the classified release of the Stego-Images-Dataset introduced by Cassavia et al. (JOWUA, 2022), which consists of 512x512 PNG images carrying payloads embedded by LSB substitution across several payload types: JavaScript, HTML, PowerShell, and Ethereum addresses. From this dataset a reproducible sample was drawn with a fixed random seed, taking fifteen images from each of five classes — the four malicious payload types plus a clean control class — for a total of seventy-five images. The dataset and the tested sample are summarized in Table 17.

The metric is deliberately not a confusion matrix. A confusion matrix measures detection — acceptance versus rejection — but an LSB payload is not expected to be rejected; it is expected to be accepted and rewritten, exactly like the appended-payload and parasite cases of the previous section. The right question is therefore whether the layer destroys the hidden channel, which is measured by comparing the least-significant-bit plane of each image before and after processing. A file is counted as neutralized when at least 2% of least-significant bits change, and as survived when fewer than 1% change. The threshold is generous on purpose: a payload that is not error-corrected is destroyed by a single flipped bit in its compressed stream, and a change of around 2% corrupts any textual header or terminator, so any measurable disturbance of the bit plane is sufficient to break a recoverable payload.

Property	Value
Dataset	Stego-Images-Dataset (classified release)
Reference	Cassavia, Caviglione, Guarascio, Manco, Zuppelli — JOWUA, Vol. 13, No. 3, 2022
Content	512x512 PNG images with LSB-embedded payloads
Payload types	JavaScript, HTML, PowerShell, Ethereum addresses
Tested sample	15 images per class x 5 classes (js, html, ps, eth, clean) = 75 images
Selection	Reproducible random sample (fixed seed)

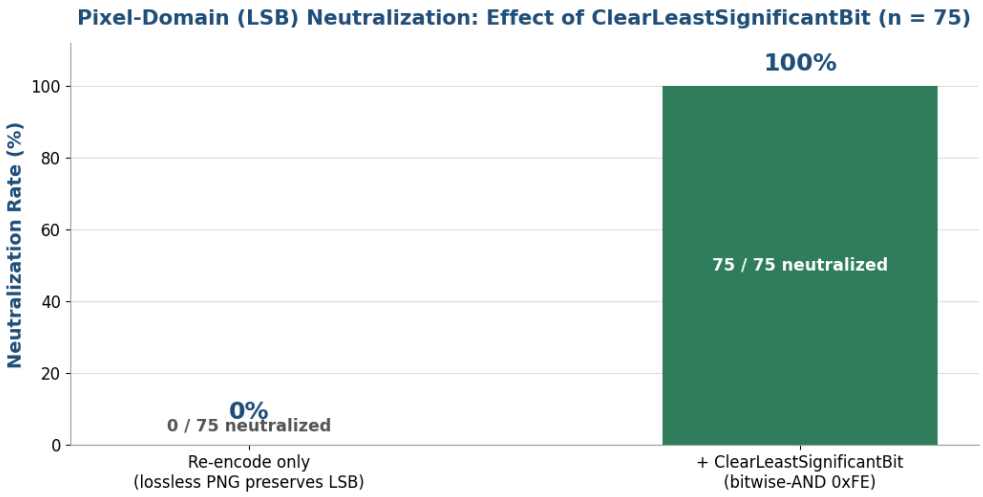
Table 17: The pixel-domain steganography dataset and the tested sample.

5.6.2 Results

With the clearing step enabled, every malicious image in the sample was neutralized: all sixty payload-carrying images across the four classes had their least-significant-bit plane disturbed well beyond the threshold, and all fifteen clean control images were accepted. The contrast with a plain re-encode is decisive. Before the clearing step was added — that is, with image re-encoding alone — none of the seventy-five images were neutralized, because lossless PNG re-encoding reproduces the pixel values bit for bit and therefore preserves the LSB payload. After the clearing step, all seventy-five are neutralized. Figure 14 shows the two conditions side by side, and the per-class breakdown is given in Table 18.

The transformation is visually imperceptible. Because only the least-significant bit is affected, each pixel changes by at most one intensity level, and the measured peak signal-to-noise ratio ranged from 49.35 dB at the lowest to 52.99 dB on average — comfortably above the 48 dB level commonly taken as the threshold of visual imperceptibility. The layer therefore destroys the hidden channel while leaving the visible image indistinguishable from the original, which is the property that makes it acceptable to apply to every accepted image rather than only to suspected ones.

Figure 14*: Pixel-domain (LSB) neutralization before and after LSB clearing*



Class	n	Neutralized	Survived	Rate
js (JavaScript)	15	15	0	100%
html	15	15	0	100%
ps (PowerShell)	15	15	0	100%
eth (Ethereum)	15	15	0	100%
Total malicious	60	60	0	100%
clean (control)	15	— (all accepted)	—	15/15 accepted

Table 18**: Per-class neutralization of LSB payloads on the tested sample.**

5.6.3 Technique and Scope

The clearing step zeroes the least-significant bit of every colour channel by a bitwise-AND with 0xFE, after first normalizing any palette-indexed image to true colour so that the operation acts on pixel intensities rather than palette indices. An earlier attempt used a posterization operator, which rounds each channel to the nearest of a reduced set of levels; that approach is not deterministic in its effect on the low bit, because rounding can leave the least-significant bit set on light pixels, and it failed to neutralize one image in testing while succeeding on the rest by coincidence. The bitwise-AND is deterministic: it clears the low bit of every pixel without exception, which is the behavior a neutralization guarantee requires.

The scope of this result is the PNG pixel domain. Steganography that hides data in the frequency domain — for example in the quantized DCT coefficients of a JPEG — is not addressed by clearing

a spatial-domain least-significant bit, and is left as future work. This bounds the claim precisely: the layer neutralizes spatial-domain LSB payloads in lossless raster images completely and imperceptibly, and does not claim to defeat transform-domain steganography. Consistent with its status as a boundary case of the upload-safety model, the step is exposed behind a configuration flag and applied as a defense-in-depth measure rather than as one of the core rejection gates.

5.7 Timing and Performance

The per-layer trace records wall-clock timings for every stage, which makes the cost structure of the pipeline visible. The structural layers are effectively free; the dominant costs are the YARA scan and, for PDFs, the recursive inflation, with image re-encoding a moderate cost that is paid only for accepted images. Ordering the layers by cost means that most rejected files never reach the expensive stages, and the content cache means that repeated uploads of the same file skip the detection layers entirely. Together these two design choices keep the amortized cost of the pipeline low even though its worst-case per-file work is dominated by the detection layers. The relative cost of the layers is depicted schematically in Figure 15.

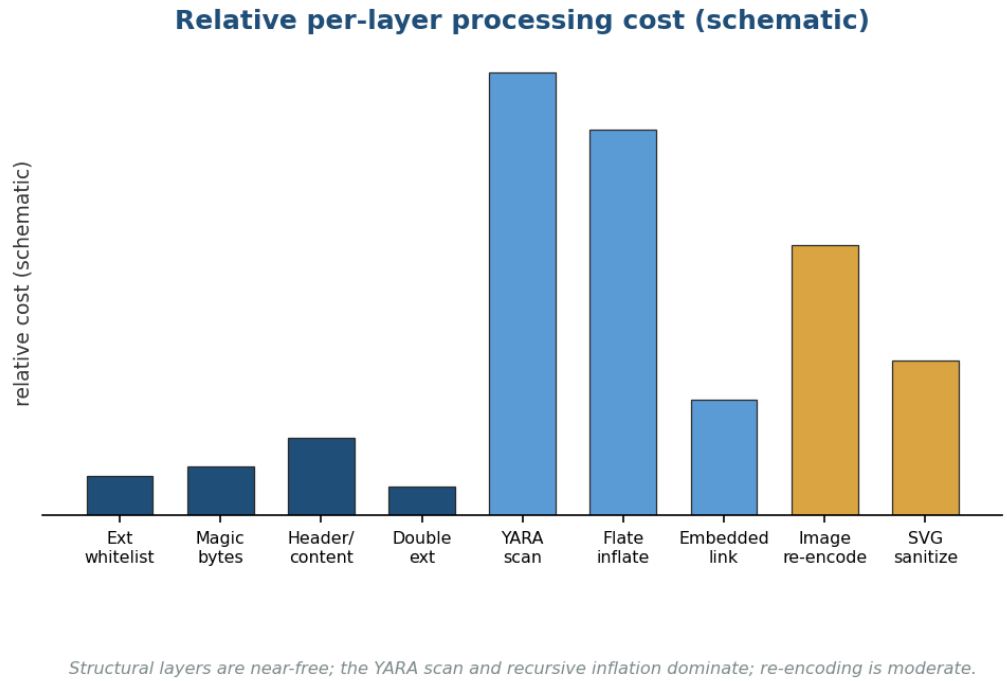


Figure 15: Relative per-layer processing cost

5.8 Contextual Comparison with Machine-Learning Detectors

Machine-learning detectors report very high accuracy on CIC-Evasive-PDFMal2022, but the comparison with SecureUploader is contextual rather than head-to-head: those systems train on pre-extracted feature tables with train/test splits, whereas SecureUploader operates on raw bytes with no training. Table 19 places the numbers side by side while making the difference in operating regime explicit.

Approach	Reported accuracy	Operating regime
ML classifier (Issakhani et al.)	~98.7%	Trained on extracted features
ML classifier (Al-Haija)	~98.84%	Trained on extracted features
SecureUploader (this work)	93.54%	Raw bytes, no training, byte-level

Table 19: Contextual comparison with machine-learning detectors on CIC-Evasive-PDFMal2022.

The point of the comparison is not to claim parity on a leaderboard but to situate a transparent, training-free, byte-level pipeline against opaque, trained, feature-level classifiers. SecureUploader trades a few points of headline accuracy for the ability to act at the point of upload, on raw bytes, with an auditable per-layer explanation and a neutralization option that a classifier does not provide. For a deployment that must justify each decision and preserve legitimate files, those properties can matter more than the last few points of accuracy.

5.9 Comparison with Related Systems

Against the closest published analog, FileUploadChecker, SecureUploader differs in two measured respects: it adds recursive inflation and re-encoding that harden it against the nested-filter and polyglot fixtures on which a conformance-only checker would pass the file, and it reports neutralization as a distinct verdict rather than a binary decision. Against the FUEL taxonomy, the adversarial corpus is organized so that its fixtures instantiate FUEL's evasion categories, so the fixture-by-fixture results in Figure 11 can be read as coverage of that taxonomy rather than of an ad-hoc set of cases.

5.10 Chapter Summary

On realistic documents the pipeline reaches 95.48% recall and 93.54% accuracy, with the false positives explained by a corpus whose clean half was engineered to resemble the malicious class

and shown to collapse from a 96.0% to an 8.87% rate once a severity gate is applied and the false negatives explained by the limits of a signature-and-structure approach. On adversarial fixtures it is the only validator among six that rejects or neutralizes every input, because each evasion class has a dedicated layer. The per-sample analysis isolates the contribution of recursive inflation, image re-encoding, and caching; the timing discussion explains why the amortized cost stays low; and the contextual comparisons clarify what the pipeline trades and what it gains relative to learned detectors and to the closest published system.

Chapter Six — Discussion

6.1 Introduction

This chapter interprets the results. It explains why layering resists evasion and why the ordering of the layers matters, documents the three self-bypass vulnerabilities found and fixed during development, discusses the distinction between detection and neutralization and the value of the evidence trail, compares SecureUploader qualitatively with prior approaches, and states the limitations, the threats to validity, and the ethical considerations of working with real malware.

6.2 Why Layering Resists Evasion

The adversarial results in Chapter Five follow directly from the design principle. Because each layer reasons about an independent property, an input crafted to satisfy one layer gains nothing against the others. A polyglot with a genuine image header satisfies magic-byte validation but is destroyed by re-encoding; a nested-filter PDF passes plain signature scanning but is exposed by recursive inflation; an active SVG has a valid structure but loses its script to sanitization. No single forgery satisfies all applicable layers at once, which is precisely why SecureUploader is the only validator in the comparison that handles every fixture. The comparison also shows the converse: each reference scanner is strong exactly on the properties it checks and blind everywhere else, so its coverage is the union of a few properties rather than of all of them.

6.3 The Value of Ordering

Ordering is not merely an optimization. Running the cheap structural layers first means the expensive detection and transformation layers see only well-formed files of an accepted type, so they can make format-specific assumptions safely rather than defensively re-checking structure. It also bounds the work an attacker can force the pipeline to do: a malformed or disallowed file is rejected before it can reach the inflation or re-encoding stages, so an attacker cannot cheaply drive the pipeline into its most expensive paths. The order therefore contributes to both correctness and robustness, not only to speed.

6.4 Self-Bypass Findings and Fixes

Designing each layer with its own bypass in mind surfaced three vulnerabilities in the pipeline itself during development. Each was diagnosed and fixed, and the fixes are themselves part of the contribution, because they demonstrate the adversarial mindset applied to the tool rather than only

to external attackers. The three findings are summarized in Table 20 and then discussed individually.

Finding	Mechanism	Fix
Nested-filter evasion	Payload behind multiple FlateDecode passes escaped a single-inflation scan	Recursive, bounded inflation to configurable depth (Layer 5b)
AV / scanner race	Host AV quarantined temp payloads before YARA could read them	Route scanner temp files to one excluded directory; RTP otherwise on
Stale-cache masking	A cached verdict could hide a newly-added rule's detection	Include YARA rules fingerprint in the cache key; invalidate on edit

Table 20: The three self-bypass vulnerabilities found during development and their fixes.

6.4.1 Nested-Filter Evasion

The first finding was that a payload wrapped in more than one FlateDecode pass escaped a scanner that inflated only once. This is the vulnerability that motivated Layer 5b's recursive, bounded inflation, and the pair of EICAR fixtures in Chapter Five is the direct evidence that the fix works at more than one level of nesting.

6.4.2 The Antivirus / Scanner Race

The second finding was environmental but no less real: the host antivirus quarantined the decompressed payload in the scanner's temporary file before YARA could read it, so genuine detections were recorded as failures. The fix — routing all scanner temporary files to a single excluded directory while leaving real-time protection on everywhere else — removed the interference without weakening host protection, and the episode is a reminder that a detector's measured performance is entangled with the environment it runs in.

6.4.3 Stale-Cache Masking

The third finding was that a cached verdict could hide the detection of a newly-added rule: after a rule was added, an identical file already in the cache would return its old, pre-rule verdict. The fix was to include a fingerprint of the YARA rules file in the cache key, so any change to the rules invalidates every cached entry immediately. This is why the cache key has three parts rather than two, and it is the property the cache-behavior fixture in Chapter Five verifies.

6.5 Detection versus Neutralization

Treating neutralization as a first-class outcome changes what the pipeline can safely accept. A file that is ambiguous but potentially legitimate — an image with appended bytes, an SVG with a stray handler — need not be rejected outright, denying a legitimate user, nor accepted as-is, admitting a risk. It can be rewritten into a provably inert form that preserves its visible content. Neutralization is therefore not a weaker form of rejection but a distinct capability, and measuring it as a second metric captures value that a detection-only account would miss. Its limits are equally important to state: re-encoding destroys most embedded payloads but, as prior work shows, certain pixel-level steganographic payloads can survive some round-trips, so neutralization reduces rather than wholly eliminates risk for that narrow class — which is why it complements, rather than replaces, the detection layers.

6.6 Transparency and Auditability

The per-layer evidence trail is more than a convenience. Because every verdict names the layer that acted and the evidence it acted on, a decision can be reproduced, audited, and explained to a stakeholder who is not a security specialist. This is the sharpest qualitative difference from a machine-learning detector: a classifier returns a score whose derivation is opaque, whereas SecureUploader returns a verdict accompanied by the exact reasoning that produced it. In settings where a wrong decision must be justified — a blocked legitimate upload, or an admitted malicious one — auditability is not a luxury but a requirement.

6.7 Qualitative Comparison with Prior Approaches

Figure 16 compares SecureUploader with three families of prior approach — DVWA-style validators, a basic extension-plus-magic baseline, and machine-learning classifiers — across six structural properties rather than on benchmark accuracy. SecureUploader is the only approach that scores across the whole set: it resists polyglots, handles nested-filter PDFs, neutralizes payloads, sanitizes SVG, requires no training, and produces a transparent trace. Each prior family is strong on some axes and absent on others, which is the same pattern the adversarial fixture results show, viewed now as capabilities rather than as per-fixture verdicts.

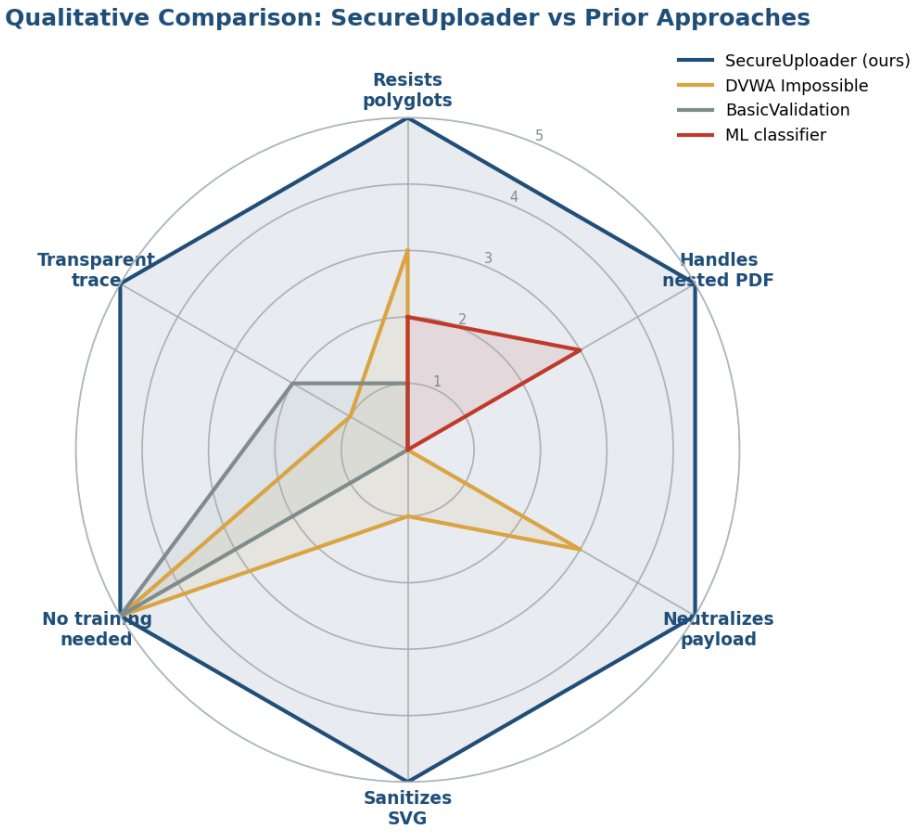


Figure 16: Qualitative comparison vs prior approaches

6.8 Limitations

- **Format scope.** The pipeline accepts a narrow set of image and PDF types; Office and archive formats, with their macro and container semantics, are out of scope and would require their own dedicated layers to handle safely.
- **Steganographic survival.** Image re-encoding alone leaves pixel-domain payloads intact on lossless formats, but the optional least-significant-bit clearing step evaluated in Section 5.6 neutralizes 100% of spatial-domain LSB payloads on the tested PNG sample while remaining visually imperceptible. The residual limit is transform-domain steganography — for example payloads hidden in JPEG DCT coefficients — which a spatial-domain LSB clear does not address and which is left for future work.
- **Encrypted PDFs.** Once the severity-threshold gate is applied, the false-positive rate on the clean set falls from 96.0% to 8.87%, and encrypted, unscannable PDFs on which the pipeline

fails closed account for most of what remains; a further downgrade of the encrypted-PDF rule is the next step for that residual.

- **Signature coverage.** Detection of malicious PDFs depends on the rule set and the structural checks; novel indicators for which no rule exists account for the false negatives and are reduced only by growing the rules and the corpus.
- **Sample availability.** Some older real polyglot hashes are unavailable through the single sanctioned download route, bounding the size of the real-sample portion of the adversarial corpus.

6.9 Threats to Validity

Following standard practice, threats to validity are separated by type and each is paired with the mitigation applied, as summarized in Table 21.

Type	Threat	Mitigation
Internal	Host AV interfered with detection measurement	Excluded temp directory; RTP otherwise on
Construct	Adversarial rates derived from documented scanner logic	Framed as evasion-resistance, not a population estimate
External	Aggregate metrics from a single PDF corpus	Adversarial corpus reported alongside the aggregate one
External	Some real samples unavailable	Logged as a data-availability limitation

Table 21: Threats to validity and the mitigation applied to each.

The construct threat deserves emphasis: the adversarial detection rates characterize behavior on crafted fixtures under each scanner's documented logic, and are best read as an evasion-resistance argument rather than an estimate over a population of real uploads. Reporting the aggregate corpus alongside them is what keeps the overall evaluation grounded in realistic files as well as in engineered ones.

6.10 Deployment Considerations

Several properties of the pipeline bear on how it would be deployed in a real system. Because it is training-free and deterministic, it requires no model to maintain, retrain, or version; the only artifact that evolves is the YARA rule set, and the cache is designed to track that evolution automatically. Because it fails closed and enforces hard resource limits, it can be placed directly

in the request path without exposing the host to unbounded work from a hostile upload, though a deployment expecting large files should tune the size and expansion limits to its own hardware.

The choice between rejection and neutralization is itself a deployment decision. A high-security context may prefer to reject any file that triggers a transformation, treating neutralization as a signal of suspicion rather than a remedy; a usability-sensitive context may prefer to neutralize wherever possible so that legitimate users are not turned away. Because the verdict and its acting layer are always recorded, either policy can be implemented on top of the same pipeline, and the decision can be revisited without changing the inspection logic. The one non-negotiable operational step is disabling the demo switch before deployment, so that the comparison endpoints used for evaluation are not exposed in production.

6.11 Ethical Considerations

Working with real malicious samples carries obligations that were treated as part of the methodology rather than as an afterthought. Samples were obtained only through a single sanctioned route and handled only under supervisor approval, within a hardened host configuration, and with automatic submission disabled so that samples were not inadvertently redistributed. The two most dangerous real polyglots were deliberately never downloaded, an inert equivalent being built instead, and that containment decision is documented. The purpose throughout was defensive — to build and evaluate a protection — and the report deliberately describes evasion techniques at the level needed to justify the defenses rather than as a construction manual.

6.12 Chapter Summary

Layering resists evasion because independent properties cannot be forged simultaneously, and ordering makes that resistance both efficient and robust. The three self-bypass findings show the principle applied reflexively to the tool; neutralization is a distinct and measurable capability with clearly-stated limits; the evidence trail makes every verdict auditable; and across six structural properties SecureUploader is uniquely complete among the compared approaches, subject to the stated limitations, threats to validity, and the ethical constraints under which real malware was handled.

Chapter Seven — Conclusion and Future Work

7.1 Summary of the Work

This project set out to build a file-upload defense that resists the evasion techniques which defeat single-check validators, and to treat neutralization as a measurable outcome alongside detection. The result is SecureUploader, a nine-layer, byte-level, training-free pipeline that rejects, neutralizes, or accepts an untrusted file and records an auditable per-layer trace for every verdict. It reaches 95.48% recall and 93.54% accuracy on the full 10,012-file public corpus and is the only validator among six that rejects or neutralizes every fixture in a hand-built adversarial corpus.

7.2 Revisiting the Research Questions

The five research questions posed in Chapter One can now be answered.

- **RQ1 (evasion resistance).** Yes. Because each layer checks an independent property, the pipeline is the only validator among six that rejects or neutralizes every adversarial fixture, covering polyglots, nested-filter concealment, and active SVG content.
- **RQ2 (aggregate performance).** The pipeline reaches 95.48% recall, 93.05% precision, 93.54% accuracy, and a 94.25% F1 score on the full 10,012-file public corpus.
- **RQ3 (comparison).** Fixture by fixture, metadata-trusting validators catch nothing, extension-plus-image checks catch a minority, image re-encoding handles image polyglots only, and SecureUploader handles them all.
- **RQ4 (neutralization).** Neutralization adds a third outcome that preserves legitimate files while removing payloads, measurable as a distinct verdict. Plain re-encoding leaves pixel-level (LSB) steganographic payloads intact on lossless formats, but adding a deterministic least-significant-bit clearing step neutralizes 100% of the LSB payloads in a dedicated 75-image evaluation while remaining visually imperceptible (PSNR above 49 dB); the residual limit is that this result is established for the PNG pixel domain, with JPEG-domain steganography left for future work.
- **RQ5 (self-bypass).** Yes. Three self-bypass vulnerabilities — nested-filter evasion, an antivirus race, and stale-cache masking — were found and fixed.

7.3 Contributions

- **Integration.** An ordered, byte-level pipeline that composes structural, detection, and transformation layers behind a single abstraction into a defense whose layers must all be satisfied.
- **Evasion resistance.** Explicit hardening against nested-filter PDF concealment, image polyglots, and active SVG content, validated on crafted fixtures and organized around an external evasion taxonomy.
- **Neutralization as a second metric.** Deterministic re-encoding and sanitization treated as a first-class verdict and measured alongside detection, with its limits stated honestly.
- **Empirical evaluation.** Aggregate metrics on a large public corpus and a fixture-level comparison against five reference validators, with false-positive and false-negative analyses.
- **Reflexive security.** Three self-bypass vulnerabilities in the pipeline found and fixed, demonstrating the adversarial mindset applied to the tool itself.

7.4 Immediate Next Steps

- **Downgrade the encrypted-PDF rule.** With the severity-threshold gate now applied — rejecting only high- and critical-severity findings, which cut the clean-set false-positive rate from 96.0% to 8.87% — the remaining residual is dominated by encrypted, unscannable PDFs; downgrading that rule is the next step for removing it.
- **Complete the adversarial run.** Grow the adversarial corpus toward its target size and run the full comparison through the evaluation endpoint to produce detection, false-positive, neutralization, and timing numbers at scale.
- **Ablation study.** Disable each layer in turn to quantify its marginal contribution to detection and neutralization, turning the qualitative coverage map into a measured one.

7.5 Longer-Term Directions

- **Broaden the format scope.** Extend neutralization to additional formats whose structure can be safely rebuilt, each with its own dedicated layer and the same evasion-resistance discipline.
- **Strengthen anti-steganography.** Investigate re-encoding strategies that further reduce the narrow class of pixel-level payloads able to survive a round-trip.

- **Policy-driven verdicts.** Allow deployments to express their own risk tolerance as configuration, choosing per-format between rejection and neutralization.
- **Complementary learned layer.** Explore adding an optional learned detector as an additional layer for the residual false negatives, without sacrificing the transparency of the byte-level core.

7.6 Reflections and Lessons Learned

Two lessons stand out from the engineering. First, the most valuable defensive insight came from attacking the tool itself: each of the three self-bypass findings improved the design more than any external test did, which argues for making the self-bypass mindset a routine part of building a detector rather than a final check. Second, the measurement environment is part of the experiment: the antivirus race showed that a detector's reported performance can be silently distorted by the host it runs on, so controlling and documenting that environment is as important as the detector's own logic.

7.7 Concluding Remarks

File upload will remain a high-value target as long as servers must judge attacker-controlled bytes. SecureUploader shows that composing a small number of independent, well-ordered layers — and adding the ability to rewrite a file into an inert form — yields a defense that resists evasion far better than any single check, while remaining transparent about every decision it makes. The gap between a leaderboard classifier and a deployable, auditable, training-free defense is exactly the gap this project set out to close.

References

[1] OWASP Foundation. Unrestricted File Upload and File Upload Cheat Sheet. OWASP, 2023.

[2] OWASP Foundation. OWASP Top 10 Web Application Security Risks. OWASP, 2021.

[3] FileUploadChecker: Server-Side Validation of Uploaded Files. Proceedings of ARES 2022.

[4] Neef, S., & Oudeh, M. FUEL: A Framework for Evaluating File-Upload Vulnerabilities. DIMVA 2024. arXiv:2405.16619.

[5] Koch, S., et al. On the Abuse and Detection of Polyglot Files. ACM The Web Conference (WWW) 2025. arXiv:2407.01529.

[6] Issakhani, M., et al. PDF Malware Detection Based on Stacking Learning (CIC-Evasive-PDFMal2022). ICISSP 2022. DOI:10.5220/0010908400003120.

[7] Al-Haija, Q. A. Machine-Learning Detection of Malicious PDF Documents. 2022.

[8] Belkind, A., Dubin, R., & Dvir, A. Open-Set Image Round-Trip Sanitization and Steganographic Survival. ICDR 2023. arXiv:2307.14057.

[9] Albertini, A. Mitra: A Tool for Generating Polyglot Files. Open-source, 2022.

[10] Albertini, A. Corkami: Studies of File Formats and Polyglots. Open-source project.

[11] VirusTotal / YARA. YARA: The Pattern Matching Swiss Knife for Malware Researchers. Documentation, 2024.

[12] abuse.ch. MalwareBazaar: Malware Sample Sharing Platform. API documentation, 2024.

[13] ImageMagick / Magick.NET. Magick.NET Documentation. 2024.

[14] Adobe Systems. PDF Reference, Sixth Edition (ISO 32000-1). 2008.

[15] W3C. Scalable Vector Graphics (SVG) 1.1 Specification. 2011.

[16] W3C. XML External Entity (XXE) Processing and Security Considerations. 2023.

[17] EICAR. Standard Anti-Virus Test File. EICAR, 2006.

[18] Microsoft. ASP.NET Core Middleware and Request Pipeline. .NET 8 Documentation, 2024.

[19] Microsoft. .NET 8 Runtime and Language Reference. 2024.

[20] Damn Vulnerable Web Application (DVWA). File Upload Module Documentation. 2023.

[21] Kaggle. Natural Images Dataset (P. Roy). 2018.

[22] MITRE. CWE-434: Unrestricted Upload of File with Dangerous Type. 2023.

[23] MITRE. CWE-79: Improper Neutralization of Input During Web Page Generation (Cross-Site Scripting). 2023.

[24] NIST. National Vulnerability Database (NVD). 2024.

[25] Stevens, D. Analyzing Malicious PDF Documents. Technical notes, 2011.

Appendices

Appendix A — Representative YARA Rules

The following rules are representative of the twenty-rule set used by the signature-scanning layer. They are shown to illustrate the form of the rules — a set of string patterns and a boolean condition over them — rather than to reproduce the full rule file. The EICAR rule matches the standard antivirus test string; the web-shell rule flags a PHP opener combined with a command-execution primitive; and the PDF-action rule flags the combination of an automatic action with embedded document script.

```
rule EICAR_Test_File
{
    strings:
        $eicar = "X5O!P%@AP[4\\PZX54(P^)7CC)7}$EICAR-STANDARD-..."
    condition:
        $eicar
}

rule PHP_Web_Shell
{
    strings:
        $php  = "<?php"
        $ex1  = "system("
        $ex2  = "shell_exec("
        $ex3  = "passthru("
    condition:
        $php and any of ($ex*)
}

rule PDF_AutoAction_JavaScript
{
    strings:
        $oa = "/OpenAction"
        $js = "/JavaScript"
    condition:
        $oa and $js
}
```

Listing 16: Three representative YARA rules from the signature-scanning layer.

Appendix B — Example Per-Layer Trace

The pipeline attaches a per-layer trace to every verdict. The two examples below are an illustrative representation of the trace format — one rejection and one neutralization — showing the fields recorded for each layer. The timing fields are illustrative of the format and are not measured values.

```
{
  "verdict": "Rejected",
  "actingLayer": "PdfFlateDecode",
  "trace": [
    { "layer": "ExtensionWhitelist",    "result": "Pass"   },
    { "layer": "MagicBytes",           "result": "Pass"   },
    { "layer": "HeaderContentMatching", "result": "Pass"   },
    { "layer": "DoubleExtension",       "result": "Pass"   },
    { "layer": "SignatureScanning",     "result": "Pass"   },
    { "layer": "PdfFlateDecode",        "result": "Reject",
      "reason": "EICAR signature exposed at inflation depth 2" }
  ]
}
```

Listing 17: Illustrative trace for a rejected file (nested-filter payload).

```
{
  "verdict": "Neutralized",
  "actingLayer": "ImageReEncoding",
  "fromCache": false,
  "trace": [
    { "layer": "ExtensionWhitelist", "result": "Pass" },
    { "layer": "MagicBytes",         "result": "Pass" },
    { "layer": "SignatureScanning",  "result": "Pass" },
    { "layer": "ImageReEncoding",    "result": "Neutralize",
      "reason": "re-encoded from pixels; 28554 -> 14863 bytes" }
  ]
}
```

Listing 18: Illustrative trace for a neutralized file (appended payload).

Appendix C — Adversarial Corpus Composition

The adversarial corpus targets roughly 155 files, balanced between a malicious/evasive portion and a clean portion so that both detection and false positives can be measured. Its planned composition by category is given below; the exact counts of the real-sample rows depend on sample availability through the single sanctioned download route, and any unavailable hashes are logged as a data-availability limitation.

Portion	Category	Approx. count	Source / notes
Malicious	Real polyglots (2 filename variants each)	24	Koch et al. catalog, by hash
Malicious	Malicious / evasive PDFs	30	CIC-Evasive-PDFMal2022
Malicious	Real active SVGs	8–10	script / handler / XXE cases
Malicious	Crafted inert fixtures	8	Mitra + hand-built
Clean	Natural JPEG images	40	Kaggle natural-images
Clean	Benign PDFs	25	ordinary documents
Clean	Sanitized CVs	5	document variety
Clean	Legitimate SVGs	8	no active content
Clean	WebP / PNG images	5	format coverage

The crafted inert fixtures deliberately reproduce evasion structures without any live payload — for example a JPEG-comment parasite, a PNG text-chunk parasite, a triple-FlateDecode PDF, an expansion-bomb PDF, SVGs exercising script, onload, foreignObject, and external-entity cases,

and a WebP with appended bytes — so that each evasion class is represented by an input whose expected verdict is known in advance.

Appendix D — Configuration Example

The pipeline is configured through the sections summarized in Chapter Four. An illustrative configuration is shown below. Note that the demo switch, which gates the evaluation-only comparison endpoints, is set to false — its required state for a production deployment.

```
{
  "Upload":     { "AllowedExtensions":
                    [".jpg", ".jpeg", ".png", ".webp", ".svg", ".pdf"],
                  "MaxSizeBytes": 10485760 },
  "Yara":       { "RulesPath": "rules/secureuploader.yar" },
  "PdfDecode":  { "MaxDepth": 8, "MaxExpandedBytes": 52428800 },
  "ReEncoding": { "MaxPixels": 40000000 },
  "Cache":      { "Enabled": true },
  "Demo":       { "Enabled": false }
}
```

Listing 19: An illustrative configuration (demo mode disabled for production).

Appendix E — Reference-Scanner Logic

For completeness, the reference validators used in the adversarial comparison are re-implementations, in C#, of publicly documented validation logic. The DVWA levels model the four documented behaviors of that application's file-upload module — no validation, client content-type trust, an extension-plus-image-size probe, and an extension-whitelist-plus-re-encode — while the basic baseline models a common extension-plus-magic-byte check. DVWA itself is not executed; only its documented logic is reproduced, so that the comparison isolates validation behavior rather than any incidental property of a running application. The full logic of each is summarized in the reference-scanner table in Chapter Five.

Appendices

Appendix A — Representative YARA Rules

The following rules are representative of the twenty in the signature-scanning layer. They are shown to illustrate the structure and intent of the rule set; the full set is maintained alongside the source.

```
rule EICAR_Test_File
{
    strings:
        $eicar = "X5O!P%@AP[4\\PZX54(P^)7CC)7}$EICAR"
    condition:
        $eicar
}
```

Listing 20: The EICAR test-file rule used to validate the detection path end-to-end.

```
rule PHP_Web_Shell
{
    strings:
        $php  = "<?php"
        $sys1 = "system("
        $sys2 = "shell_exec("
        $sys3 = "passthru("
    condition:
        $php and any of ($sys*)
}
```

Listing 21: A rule flagging PHP web-shell markers carried inside an image polyglot.

```
rule PDF_OpenAction_JavaScript
{
    strings:
        $oa = "/OpenAction"
        $js = "/JavaScript"
    condition:
        $oa and $js
}
```

Listing 22: A rule flagging a PDF that runs JavaScript automatically on open.

Appendix B — Example Per-Layer Traces

The pipeline attaches a per-layer trace to every verdict. The two examples below illustrate the format of that trace for a rejected file and for a neutralized file. They are format examples: the field values, including the per-layer timings, are illustrative rather than measured.

A rejected PDF whose payload is exposed only after two levels of inflation:

```
{
  "verdict": "Rejected",
  "actingLayer": "PdfFlateDecode",
  "trace": [
    { "layer": "ExtensionWhitelist",    "result": "Pass"   },
    { "layer": "MagicBytes",            "result": "Pass"   },
    { "layer": "HeaderContentMatching", "result": "Pass"   },
    { "layer": "DoubleExtension",       "result": "Pass"   },
    { "layer": "SignatureScanning",     "result": "Pass"   },
    { "layer": "PdfFlateDecode",        "result": "Reject",
      "reason": "EICAR signature at inflation depth 2" }
  ]
}
```

Listing 23: Trace format for a file rejected by recursive inflation.

A neutralized image whose appended payload is removed by re-encoding:

```
{
  "verdict": "Neutralized",
  "actingLayer": "ImageReEncoding",
  "trace": [
    { "layer": "ExtensionWhitelist", "result": "Pass"        },
    { "layer": "MagicBytes",         "result": "Pass"        },
    { "layer": "SignatureScanning",  "result": "Pass"        },
    { "layer": "ImageReEncoding",    "result": "Neutralize",
      "reason": "re-encoded from pixels; 28554 -> 14863 bytes",
      "fromCache": false }
  ]
}
```

Listing 24: Trace format for a file accepted after neutralization.

Appendix C — Adversarial Corpus Composition

The adversarial corpus targets roughly 155 files, split between malicious or evasive inputs and clean inputs used to measure false positives. Its composition is summarized below; real polyglot samples are drawn by hash from the Koch et al. catalog and retrieved through a single sanctioned route, while inert fixtures are generated locally.

Category	Count	Source / notes
Real polyglots (2 filename variants each)	~24	Koch et al. catalog, by hash
Malicious PDFs	~30	CIC-Evasive-PDFMal2022
Real malicious SVGs	~8–10	Active-content samples
Crafted inert fixtures	~8	Mitra-generated evasion cases
Benign JPEGs	~40	Kaggle natural-images
Benign PDFs	~25	Clean document set
Sanitized CVs	~5	Clean, realistic documents
Legitimate SVGs	~8	Clean vector graphics
Benign WebP / PNG	~5	Clean raster images

The crafted inert fixtures include a JPEG comment-segment parasite, a PNG text-chunk parasite, a triple-FlateDecode PDF, an expansion-bomb PDF, SVGs carrying a script element, an onload

handler, a foreignObject node, and an external-entity declaration, and a WebP with appended bytes. Each fixture's expected verdict is known in advance, which is what makes the adversarial evaluation unambiguous.

Appendix D — Example Configuration

The pipeline is configured through the sections described in Chapter Four. An illustrative configuration is shown below; note that the demo switch, which exposes the comparison endpoints used for evaluation, is set to false as it must be before deployment.

```
{
  "Upload": {
    "AllowedExtensions": [".jpg", ".jpeg", ".png", ".webp", ".svg", ".pdf"],
    "MaxSizeBytes": 10485760
  },
  "Yara":       { "RulesPath": "rules/secureuploader.yar" },
  "PdfDecode":  { "MaxDepth": 8, "MaxExpandedBytes": 52428800 },
  "ReEncoding": { "MaxPixels": 40000000 },
  "Cache":      { "Enabled": true },
  "Demo":       { "Enabled": false }
}
```

Listing 25: An example appsettings configuration with the demo switch disabled.

Appendix E — Reference-Scanner Logic

The five reference validators against which SecureUploader is compared were re-implemented in C# from their documented logic so that the identical bytes could be run through each. Their behavior is summarized here for reference; the DVWA scanners reproduce the documented logic of the corresponding DVWA security levels, and the basic baseline performs an extension plus magic-byte check. DVWA itself is not executed — only its documented validation logic is reproduced, so the comparison reflects the design of each level rather than a particular deployment.

Scanner	Reproduced logic
DVWA Low	No validation; accepts any uploaded file
DVWA Medium	Trusts the client-supplied Content-Type header
DVWA High	Extension check plus an image-size probe
DVWA Impossible	Whitelist, size probe, and image re-encode
BasicValidation	Extension whitelist plus magic-byte match

Appendix F — Adversarial Fixtures and Expected Verdicts

Each adversarial fixture has a known expected verdict, which is what makes the fixture-level evaluation unambiguous. The representative fixtures and their expected outcomes are listed below, grouped by the evasion class each one exercises.

Fixture	Evasion class	Expected verdict	Acting layer
PHP renamed .png (no header)	Extension spoofing	Reject	MagicBytes (2)
Script disguised as .jpg.png	Double extension	Reject	DoubleExtension (4)
JPEG + appended ZIP	Zipper polyglot	Neutralize	ImageReEncoding (6)
PNG + PHP (parasite)	Parasite polyglot	Neutralize	ImageReEncoding (6)
JPEG COM-segment parasite	Parasite polyglot	Neutralize	ImageReEncoding (6)
WebP + appended bytes	Cavity / appended	Neutralize	ImageReEncoding (6)
EICAR single FlateDecode	Nested filter (d1)	Reject	PdfFlateDecode (5b)
EICAR double FlateDecode	Nested filter (d2)	Reject	PdfFlateDecode (5b)
Triple FlateDecode payload	Nested filter (d3)	Reject	PdfFlateDecode (5b)
Expansion-bomb PDF	Decompression bomb	Reject	PdfFlateDecode (5b)
PDF with /Launch action	Embedded action	Reject	EmbeddedLink (5c)
SVG with script element	Active SVG	Neutralize	SvgSanitization (7)
SVG with onload handler	Active SVG	Neutralize	SvgSanitization (7)
SVG with foreignObject	Active SVG	Neutralize	SvgSanitization (7)
SVG with external entity	XXE / SSRF	Neutralize	SvgSanitization (7)

Appendix G — YARA Rule Inventory

The signature-scanning layer carries twenty rules across six categories. The inventory below names the categories and the indicators each targets; the rule bodies for three representative rules appear in Appendix A.

Category	Rules	Targeted indicators
Test markers	1	EICAR standard test string
Web shells	5	PHP, ASP, JSP shell tokens and command sinks
Script injection	4	script elements, event-handler attributes
PDF actions	4	JavaScript, OpenAction, Launch, embedded-file actions
Archive markers	3	embedded ZIP / RAR / 7z signatures
Obfuscation	3	suspicious encodings and high-entropy regions